FIT2099 Assignment 3 Design Rationale

Requirement	Author
REQ 1: Second Spring	Poyee Tan 36641162
REQ 3: Infestation and Pest Control	Jingyu Zhang 37033956
REQ 4: Enchanted Tools	Yuanshu Liu 36226173
REQ 5: The Wanderer (API, HD)	Jingyu Zhang, Yuanshu Liu, Poyee Tan

Poyee Tan 36641162 – REQ 1: Second Spring

REQ1 makes plants alive. Every plant ages one turn at a time. Each turn it tries its abilities (grow, spread, produce) in order, and does at most one thing. Each young stage is its own class, and only tilled Soil accepts new plants. We had three design goals. The shared timing rule is written once. Each plant keeps its own rules in its own class. Other features (pests, enchantments, weather) act on plants without naming any crop. Abstractions were added only where the brief already shows variation between nine plants or three abilities (KISS, YAGNI).

A. One class per growth stage

The brief says a seedling grows into a sprout, and a sprout into a crop. It also says growing creates "a brand new plant with age 0".

Design 1: one class per crop with a `stage` field and a `switch` that picks the character, abilities and rules.

Pros	Cons
One class holds a crop's whole life, so a reader sees every stage in one file (KISS).	Every new stage would mean editing every `switch` on the stage field (OCP, the Switch Statements smell).
The class diagram gains no new classes, so the design stays small on paper.	One class would own several display characters, ability lists and harvest rules, so it changes for many reasons (SRP).
	A sprout would still offer watering and harvesting, guarded by `if (stage == SPROUT)` checks scattered through the class.

Design 2: one class per crop holding a separate stage object (State pattern) that it swaps as it grows.

Pros	Cons
The crop stays the same object while it grows. Anything that holds it still sees it.	The brief says growing creates a new plant with age 0. So every swap would have to reset the age by hand (Connascence of Execution).
Each stage's rules still sit in their own state class, so the stage logic stays separate.	The ground object would have to change its character, name and allowed actions at run time. The engine's `Ground` is not built for this, and any caller holding the old values would be out of date.

Design 3 (chosen): one `Plant` subclass per stage. `Plant` is a new abstract class between `Ground` and `Crop`, and `Crop` now extends it. Growing replaces the ground with a new object, so the new stage starts at age 0 with no reset code. The young stages extend `Plant`, not `Crop`: as `Crop`s they would inherit watering and harvesting they must refuse.

Pros	Cons
A new crop's stages are new classes, and no existing stage changes (OCP).	There are four small classes that mostly hold constants. Acceptable because each small class removes a `switch` branch that would otherwise sit in one large class.
Every `Crop` really can be watered and harvested, so no sprout refuses a `Crop` method (LSP).	Stages name each other, for example `StrawberrySprout.nextStage()` returns `new Strawberry(...)` (Connascence of Name). Acceptable because the names stay inside `game.crops`, and each stage has only one next stage.
Each class has one character, one ability list and one set of rules (SRP). The age starts at 0 simply because a new object is made.	Inserting a new stage into an existing chain edits the stage before it, whose `nextStage()` must return the new class. Acceptable because only that one method changes, and a whole new crop needs no edit to existing stages.

Package placement. The young stages live in `game.crops`, beside the crops they grow into, not in `game.plants`. Otherwise `game.plants` would import `game.crops`, which imports `game.plants`: a cycle (Acyclic

Dependencies Principle). `game.plants` now depends on no crop.

Foresight: a Pumpkin and its Pumpkin Sprout are two new classes. No existing class changes except `StackQueueValleyWorld`, which places them.

Connascence: stages name their next stage (Connascence of Name), kept inside `game.crops`. The rejected State pattern would add Connascence of Execution: the age must be reset by hand on every swap. **Code smells:** avoided Switch Statements on a stage field. Also avoided Refused Bequest: a sprout does not inherit watering and harvesting it would have to refuse. Accepted: small classes that mostly hold constants.

B. How a plant acts each turn

Every turn a plant ages, then tries its abilities in priority order and does at most one thing. Three questions shape this. How are the abilities stored? How does an ability get what it creates? How are turns counted?

Abilities as a list.

Design 1: each plant overrides `tick` with its own if-else.

Pros	Cons
Each plant's rules sit in one place, with no shared class to learn before reading a plant (KISS).	Nine classes would each copy the rule "every N turns, roll a chance, stop at the first success" (Duplicate Code). A bug fix would need nine edits (Connascence of Algorithm).
	One plant could easily break the "one thing per turn" rule. Nothing would stop it.

Design 2: use the engine's `Behaviour<Plant, String>` as the ability type.

Pros	Cons
No new interface is needed. The engine already has a type for "something that may act this turn".	It returns `null` for "nothing happened" (Connascence of Meaning). So every caller would need a `null` check.
	The type is also used by actors, so a plant ability would look like an actor's behaviour. The plant rule "`""` means nothing happened" would be lost.

Design 3 (chosen): `Plant` holds a `List<PlantAbility>` in priority order, passed to `super(...)`. `PlantAbility` has one main method, `tryPerform(plant, location)`. It returns a report, or `""` when nothing happened. `Plant.tick` ages the plant, then `attemptGrowth` tries the abilities from the top and stops at the first non-empty report. `PeriodicAbility` writes the timing once: its `tryPerform` is `final` (Template Method). It checks `age % interval`, rolls the chance, then calls the abstract `perform`. `Grow`, `SelfReplicate` and `Produce` write only their effect.

Pros	Cons
The timing rule and the loop over abilities are written once. So every plant keeps the "one thing per turn" rule (DRY).	A reader opens two classes, the plant and the ability, to see one behaviour. Acceptable because each ability is short and named after what it does.
To add an ability, such as "wither", we write one new class. No other ability changes (OCP).	Priority is the order of the constructor arguments (Connascence of Position). Acceptable because each plant's class comment lists its abilities in priority order.
`Plant` only sees its abilities as `PlantAbility`, so it never names a concrete ability (DIP). The plant decides when to act. Each ability decides what happens (SRP).	`final tryPerform` stops an ability from changing the timing. Acceptable because an ability with different timing can implement `PlantAbility` directly.
	To use a new ability, a plant must add it to its own list. So that plant's class changes. Acceptable because choosing a plant's abilities is that plant's job, and no other plant or ability changes.

`PeriodicAbility` checks its numbers when it is built. It rejects an interval below 1, because an interval of 0 used to crash on `age % 0`. It also rejects a chance below 0 or above 100.

Growth forced by magic. REQ4's Bounty Enchantment makes a plant grow at once. At first it called `attemptGrowth`. But `attemptGrowth` goes through each ability's interval and chance. So on most turns the plant was "not due", and the magic did nothing. Now `Plant` has a second method, `forceGrowth`. It tries the same abilities, but through `forcePerform`, which skips the interval and the chance. `PeriodicAbility.forcePerform` is `final` and simply calls `perform`. Any other ability gets a default `forcePerform` that just calls `tryPerform`. The cost is that `forceGrowth` repeats the short loop in `attemptGrowth` (see Section G).

Why each plant lists its own abilities. The A2 marker asked us to pass the Merchant its goods from outside. We did not do the same for a plant's abilities. A Merchant that sells other goods is still a Merchant. But a Strawberry always drops strawberries every five turns. A plant that did something else would not be a Strawberry. It would be a different plant, so we would write a new class for it.

Foresight: a Rose that withers after 30 turns is one new `PeriodicAbility` subclass placed first in its list.

Capabilities: `Growing`, `Spreading`, `Producing`. Each ability must ask the plant what to create. `Grow` needs the next stage. `SelfReplicate` needs the offspring. `Produce` needs the item. The plant knows this, and already holds the `RandomUtils` and `AreaFactory` the new object needs.

Design 1: a `Supplier` per ability, for example `new Grow(..., StrawberrySprout::new)` (our first version).

Pros	Cons
The ability is given exactly what it creates, so it needs no interface and no knowledge of the plant's type.	After A2 feedback, the team removed method references everywhere. This would be the only place still using them.
	The supplier needed the plant's `RandomUtils` and `AreaFactory`, so each plant had to build its suppliers around its own fields.

Design 2: three methods on `Plant` whose defaults throw `UnsupportedOperationException` (our draft).

Pros	Cons
There is one abstract class to learn, and each plant overrides only the methods its abilities call.	Parsnip only produces, yet carried `nextStage()` and `offspring()` that could only throw (ISP, Refused Bequest).
	The Javadoc had to say "the default throws". Callers had to know which plants support which method (Connascence of Meaning).

Design 3: an abstract class per capability, such as `ProducingPlant`.

Pros	Cons
Shared code could live in the abstract class. For example, dropping a produced item on a free tile.	A Java class has only one parent. `Strawberry` already extends `Crop`, so it could not also extend `ProducingPlant`. A plant with two capabilities could not be built at all.
	Shared code would be stuck in one branch of the class tree. A sprout and a crop could not share it.

Design 4 (chosen): one single-method interface per capability: `Growing` (`nextStage()`), `Spreading` (`offspring()`) and `Producing` (`produce()`). Each plant implements only what it can do. An ability finds its capability with `location.getGroundAs(Growing.class)` and does nothing if it is missing, the same way pests find `Regressible`.

Pros	Cons
No plant carries a method it cannot honour, and nothing throws (ISP, no Refused Bequest).	There are three more interfaces, and every ability shares Connascence of Type with one of them. Acceptable because each interface has one method, and each replaces a method that could only throw.
`getGroundAs` is the engine's capability lookup, so abilities find what they need without any `instanceof`.	A plant given `Grow` but not `Growing` silently never grows instead of failing. Acceptable because the tests catch that mistake. Also, doing nothing is safer in play than crashing.

Pros	Cons
Abilities depend on three small interfaces, not on `Plant` subclasses (DIP). `Regressible` and `Plantable` work the same way, so a reader learns this once.	The capability methods must be `public`, as interface methods are. Acceptable because only the abilities call them.

Strictly, the throwing draft did not break LSP. No path ever reached the throw. So we describe it as an ISP problem.

Foresight: a Sunflower that drops seeds implements `Producing` and is given `Produce`. No ability changes.

Counting turns. The brief says each plant's age starts at 0 when it is placed.

Design 1: the engine's `Status` countdown.

Pros	Cons
It already exists and is tested, so no new counting code is needed. A status also removes itself when its time is up.	Statuses tick and expire independently, so they cannot express "try in priority order and stop at the first success".
	Each ability would need its own status, and the age the brief asks for would still not be stored anywhere.

Design 2: a global turn counter.

Pros	Cons
There is one number for the whole game, so every plant reads the same clock. Two plants could never disagree about the turn.	The engine has none. Adding one would need a static field, the global-state smell A2 feedback penalised.
	Every plant would depend on its value (Connascence of Value). A plant's age would have to be worked out from the turn it was placed.

Design 3 (chosen): `Plant` keeps its own `age`.

Pros	Cons
Each plant counts from 0 when placed, exactly as the brief says.	The engine updates tiles row by row. So a new plant to the right of or below its parent ages once in its first turn. Acceptable because this is engine behaviour, it shifts timing by one turn, and it breaks no rule in the brief.
There is no setter, so only the plant changes its age (encapsulation).	

Each plant gets its `RandomUtils` through the constructor. A test can pass a `FixedRandom` instead, so every run gives the same result.

Connascence: ability priority is the order of the constructor arguments (Connascence of Position), documented in each plant's class comment. It replaces Connascence of Algorithm across nine copies of the loop. Abilities share only Connascence of Type with the capabilities, where the rejected draft had Connascence of Meaning ("the default throws"). Ageing before trying abilities is Connascence of Execution, kept inside one method. A global counter would have spread Connascence of Value across every plant. **Code smells:** avoided Duplicate Code, Long Method (nine hand-written `tick` methods), global static state and functional style. Removed Refused Bequest (methods that could only throw). Accepted: the Connascence of Position above, and the engine's row-by-row ticking, which shifts a new plant's timing by one turn.

C. Capabilities other features use: `Plantable` and `Regressible`

REQ1 offers two small interfaces to other features. `Plantable` says where a plant may go. `Regressible` lets REQ3's pests set a plant back.

Where plants can grow. The brief says only tilled Soil accepts new plants.

Design 1: check `instanceof Soil` wherever something is planted.

Pros	Cons
The check is one short line. The rule "only Soil" is visible right where planting happens, so a reader sees it at once.	The same check would sit in spreading, the Merchant and the Sowing omen (Duplicate Code).
	A new plantable ground would mean editing every planting path (OCP, Shotgun Surgery). A type check is also a smell.

Design 2 (chosen): a `Plantable` interface with `plant(ground, location)`, and a `PlantableTile` condition for `Area.filter`. `Soil` implements it. `plant` takes a `Ground`, not a `Plant`, so `game.positions` does not depend on `game.plants` (no package cycle).

Pros	Cons
A new plantable ground, such as fertilised soil, only implements the interface, and every planting path accepts it (OCP).	It adds one more interface and one more condition class. Acceptable because three features plant through them, so they replace three copies of the rule.
`SelfReplicate`, the Merchant's `Crop.plantNextTo` and REQ5's Sowing omen all plant through it, so they follow one rule (DRY).	`plant` takes a `Ground`, so only Connascence of Type stops a caller planting something odd. Acceptable because taking a `Ground` is what keeps `game.positions` free of `game.plants`, and every caller passes a plant.
REQ3's mole and snail find bare Soil through the same interface, so the rule is shared across features.	

In Assignment 2 the Merchant could plant on Dirt. Now it plants only on Soil, as the brief requires.

Foresight: a Greenhouse floor that accepts plants implements `Plantable`. Every planting path accepts it at once.

Regressing, for REQ3 pests.

Design 1: pests call `Crop` methods directly.

Pros	Cons
There is no new interface to learn. A pest simply calls the crop it is standing on, so the code is easy to follow.	Pests would name every crop and stage (Connascence of Type). A new stage would mean editing every pest (OCP, Shotgun Surgery).
	Pests would reach into crop internals to decide the earlier stage (Feature Envy).

Design 2 (chosen): `Plant` implements `Regressible`, whose `regress` replaces the plant with `previousStage()`. By default a regressed plant dies and leaves `Dirt`, matching A2's `Crop.removeCrop`. Later stages override `previousStage()` to return their earlier stage.

Pros	Cons
Pests depend on `Regressible`, so `game.pests` does not import `game.crops`, and only Connascence of Name (`regress`) crosses packages (DIP).	A pest cannot treat one crop differently. Acceptable because no rule in the brief or proposal treats crops differently.
Every pest eats a new plant that extends `Plant`, with no change to the pests (OCP). Each plant decides its own earlier stage.	`previousStage()` is `protected`, so classes in the same package could call it. Acceptable because no class other than `Plant` calls it.

Connascence: only Connascence of Type on `Plantable` and Connascence of Name on `regress` cross package boundaries. **Code smells:** avoided type checks for `Soil` and Feature Envy in the pests. Also avoided Shotgun Surgery when a ground or plant is added.

D. Pausing and printing in a plant's turn

This section covers two needs. REQ5's frost must stop a plant acting. And the plant must print what it did, although `Ground.tick` returns nothing.

Pausing growth, for REQ5. The proposal says that during a frost every plant around the Wanderer "stops growing".

Design 1: the status removes the plant's abilities and puts them back when it expires (push).

Pros	Cons
The plant needs no check of its own. The status does all the work.	The status would reach into `Plant`'s private list and must restore exactly the same list (Connascence of Identity).
	Two overlapping frosts could restore the list twice, or restore it while the other frost still holds.

Design 2: `Plant.tick` checks for `GrowthPausedStatus` before calling `attemptGrowth` (our first version).

Pros	Cons
The check sits at the start of the plant's turn, where a reader looks first. Deciding whether to act stays the plant's job (SRP).	Any other caller of `attemptGrowth` skipped the check, so the rule held only for the plant's own turn.
	REQ4's Bounty did not go through `tick`. So a frozen plant still grew when an enchanted watering can touched it. The proposal says it "stops growing".

Design 3 (chosen): the methods that try a plant's abilities check for `GrowthPausedStatus` themselves. There are two of them: `attemptGrowth` (the plant's own turn) and `forceGrowth` (Bounty's magic). While the plant is paused, both return `""`. `Plant.tick` still ages the plant first.

Pros	Cons
Every way into a plant's abilities passes the check: the plant's own turn and Bounty's magic. Bounty does not need to know about frost at all.	`GrowthPausedStatus.applyEffect` is empty on purpose, which a reader may see as a do-nothing override (it looks like Refused Bequest). Acceptable because a comment says why. The status is only a flag with a countdown, and `TimedStatus` (A2) already does the countdown.
The status only counts down. Deciding whether to act stays the plant's job (SRP).	The check is written twice, once in each method. Acceptable because it is one line, and both copies sit in `Plant`. A test checks that Bounty does not grow a paused plant.

While paused, the plant still ages. A test checks that Bounty does not grow a paused plant.

Printing without a Display parameter. `Ground.tick` returns nothing, so a plant prints its own report.

Design 1: a `private static final Display` (our draft).

Pros	Cons
One object is shared by every plant, so no plant creates its own. Every plant prints through the same `Display`.	Shared static state is the global-state smell A2 feedback penalised, and every plant would then depend on one hidden object. Tests would share it too, so one test's setup could leak into another.

Design 2: a `Display` injected through the constructor.

Pros	Cons
A test could pass a `Display` that records the printed text. The plant would also stop creating its own `Display` (DIP).	Nine constructors, every `nextStage()` and `offspring()`, `StackQueueValleyWorld` and every test would gain a parameter (Tramp Data).
	`Display` is the engine's console writer with no alternative to swap in, so the extra parameter would never vary.

Design 3: return the report and let someone else print it.

Pros	Cons
There is no `Display` inside `Plant` at all. The plant would only describe what happened, and someone else would print it.	Only the engine calls `Ground.tick`, and it ignores any result. So the report would be lost, and the player would never see a plant grow.

Design 4 (chosen): a `private final Display` per plant.

Pros	Cons
There is no static state. Each plant prints with one short call, and nothing is passed around (KISS).	Each plant creates its own `Display`, a small DIP gap. Acceptable because `Display` has no alternative to swap in, and the collaborators that do vary, `RandomUtils` and `AreaFactory`, are injected.
Plants share nothing, so there is no Connascence of Value between them.	

Connascence: avoided Connascence of Identity, because the status never puts a list back. Also avoided Connascence of Value between plants, because there is no shared static `Display`. The pause rule is Connascence of Execution (check before trying abilities), kept inside one method. **Code smells:** avoided a static field (global state) and a rule that one caller could skip. Accepted: an empty `applyEffect`, explained by a comment. Also a small DIP gap, because `Display` has nothing to swap in.

E. Visibility, `final`, `protected` and `static`

Choice	Where	Why	Cost
Every field is `private`	All REQ1 classes	Details stay inside the class (A2 feedback). No other class can change its data, so no class is a Data Class. Connascence on state stays inside.	Each plant keeps its own `RandomUtils` and `AreaFactory` to pass to the plants it creates.
Constants are `private static final`	e.g. `GROW_INTERVAL`, `PRICE`	A name explains each number. Java does not allow instance fields in `super(...)`, so the names must be static.	Tests repeat the numbers, because they cannot read private constants.
No static methods and no mutable static fields	Every REQ1 class	A2 feedback penalised static methods as "function thinking" and static state as global state.	`RandomUtils` and `AreaFactory` must be passed through constructors instead.
`protected` constructors	`Plant`, `PeriodicAbility`	They are abstract, so only subclasses construct them, and `protected` says so.	Same-package classes could call them, but Java forbids creating an abstract class anyway.
`protected` hooks	`perform`, `previousStage`	Subclasses override them. Callers must go through the template.	Same-package classes could call them. None does.
`final` methods	`tryPerform`, `forcePerform`, `plantNextTo`, `noRoomToPlant`	They fix the shared rule, so a subclass cannot change it.	A plant needing different timing implements `PlantAbility` directly.
`public` capability methods	`nextStage`, `offspring`, `produce`, `attemptGrowth`, `forceGrowth`	Interface methods must be public. `forceGrowth` is called by REQ4's Bounty.	Any class could ask a plant for its next stage. Only the abilities do. Both growth methods obey the pause.
Classes are not `final`	e.g. `Grow`, `Soil`	Not being `final` only makes a subclass possible. The design is open for extension because new classes plug into the abstractions (OCP).	A subclass could break a rule. Private fields and `final` templates limit the damage.

Connascence: `private` fields keep connascence on state inside each class. **Code smells:** avoided public attributes and missing modifiers.

F. Changes to Assignment 2

Change	Why	Cost
`Crop` extends `Plant` instead of `Ground`	Every crop gets an age, abilities and `Regressible`.	`Crop` constructors take abilities. Watering and harvesting are unchanged.
`Unplanted` checks for any `Plant`	Watering a Cauliflower used to flood sprouts into puddles.	One condition class now depends on `Plant`, which it uses only through `getGroundAs`.
The Merchant plants through `PlantableTile`	The brief says only Soil accepts plants.	A Merchant surrounded by Dirt cannot sell crops. The message says so.
`RandomUtils` and `AreaFactory` are injected instead of static	A2 feedback. Tests pass `FixedRandom`.	Every plant constructor has two more parameters (Tramp Data).
Fixed farm layout with one of every plant	Every plant is visible when the game starts, so a marker can check each one.	The map no longer starts empty, so the A2 opening looks different.
Unique characters: Crow `b`, Cauliflower `K`	The brief gives `v` to the Strawberry Sprout, and `C` is the Cow.	Players of A2 see new characters.

G. Known minor violations and why we accept them

Violation or cost	Why it is acceptable
`SelfReplicate` and `Crop.plantNextTo` repeat the two lines that find an adjacent Plantable tile and plant on it. DRY is not fully met.	They build different messages. A third use would be the signal to extract it.
`Crop` also builds the Merchant's "no room to plant" message, mixing trading into a ground (SRP).	It keeps one wording for both purchasable crops, Tomato and Parsnip.
Each plant creates its own `Display` (Section D).	Injecting it adds parameters everywhere without making anything replaceable (KISS).
A plant given an ability without the matching capability does nothing (Section B).	Safer than a crash. Tests check every plant's abilities.
The engine updates tiles row by row. So a new plant to the right of or below its parent ages once in its first turn.	Engine behaviour. It shifts timing by one turn and breaks no rule.
`StackQueueValleyWorld` registers the map characters with method references, `Dirt::new` (base code) and `Soil::new` (REQ1), although the team avoids functional style.	The engine's `DefaultGroundCreator.registerGround(char, Supplier<? extends Ground>)` needs a `Supplier`, and the engine cannot be changed. A named `Supplier` class for each ground would be used once, only to avoid one line (KISS). These are the only two method references in the game code.
`attemptGrowth` and `forceGrowth` repeat the same short loop and the same pause check. DRY is not fully met.	They differ in one call: `tryPerform` or `forcePerform`. Joining them would need a `boolean forced` parameter, which is the Flag Argument smell. Two copies of a five-line loop, side by side in one class, are easier to read.

Testing

`GrowTest`, `SelfReplicateTest`, `ProduceTest`, `PlantStateTest`, `PeriodicAbilityTest` and `PlantCapabilityTest` cover every plant at the right age and one turn early. They also cover a failed chance, no free tile, a paused plant and a regressed plant. Invalid cases include bad intervals and chances, and an ability on a plant that lacks the capability. `FixedRandom` makes them repeatable, and `mvn test` runs them all.

Summary

The final design uses the chosen design of every part. Each stage is one class (A). Abilities sit in an ordered list, with the timing written once and capabilities found by interface (B). `Plantable` and `Regressible` serve other

features (C). Every caller passes the pause check (D). Each abstraction answers variation the brief already shows: nine plants, three abilities and two other features that act on plants. The main cost is more, smaller types, and Connascence of Position for ability priority. In return, the design passes a real OCP test. A Pumpkin needs one new class per stage. A withering Rose needs one new `PeriodicAbility`. Neither changes `Plant`, `PeriodicAbility`, any other plant, any pest or any enchantment. Only `StackQueueValleyWorld` places them.

Principle	Where it shows	Change it absorbs	Where it is only partly met, and why
SRP	Timing, each effect, planting, regressing and pausing each have one class.	A new timing rule edits only `PeriodicAbility`.	`Crop` holds one Merchant message (Section G).
OCP	Adding a stage, ability, capability or plantable ground means writing one new class.	A Pumpkin, a wither ability, a Greenhouse floor: no existing plant, ability or planting path changes.	New plants are placed in `StackQueueValleyWorld`, the composition root.
LSP	Every `Crop` can be watered and harvested. Abilities never check the plant's type.	A new stage works wherever `Plant` is expected.	A plant given an ability without its capability does nothing instead of failing. Tests check every plant.
ISP	`Growing`, `Spreading`, `Producing`, `Plantable` and `Regressible` each have one method.	Parsnip carries only `produce()`.	`PlantAbility` has two methods, `tryPerform` and `forcePerform`. The second has a default, so an ability only writes it if it has its own gate. `Plant` also offers `attemptGrowth`, `forceGrowth`, `getAge` and `regress`, each used by a different feature.
DIP	Abilities depend on capabilities, pests on `Regressible`, `Plant` on `PlantAbility`. `RandomUtils` and `AreaFactory` are passed in through constructors, which lets tests replace them. Both are concrete classes, so this is dependency injection rather than full DIP.	A new plant or pest needs no change to the other side.	Each plant creates its own engine `Display`. Stages create their next stage directly (one outcome each, so a factory would be a parameter nobody varies).

DRY and KISS. Each rule exists once: the timing, the priority loop, the pause check, planting and the countdown. Each new interface and abstract class is used by at least two classes. **Connascence.** The connascence we found between packages is of Name and Type, through `Regressible`, `Plantable` and the capability interfaces. Stronger forms (Position for ability priority, Execution for age before abilities and the pause check) stay inside one class each.

Jingyu Zhang 37033956 – REQ 3: Infestation and Pest Control

There are three pests: `SlimeSnail`, `LocustSwarm` and `BurrowingMole`. Each implements `Infestation`. They move towards plants and set them back a growth stage. Each also leaves its own mark on the farm. Three defences, `PestTrap`, `LadybugJar` and `SaltBarrier`, implement `PestDeterrent`: they pull pests in, hunt them, or push them out. We had three design goals. Pests and defences do not know each other's concrete classes. Each shared rule (the pest turn, the landing search, the countdown) is written once. Adding a pest or a defence means writing one new class. Abstractions were added only where the proposal already names three variants, or where code would otherwise be copied (KISS, YAGNI).

A. Two abstractions, found by capability, never by class

The proposal has two sides that act on each other: pests eat plants, and defences act on pests. Each side must find the other without naming its classes.

Interfaces or abstract classes.

Design 1: `Infestation` and `PestDeterrent` are abstract classes that every pest and every defence extends.

Pros	Cons
Shared code, such as a default report, could live in the abstract class. It would not be repeated in every pest or defence.	A Java class has only one parent. `Pest` already extends `WanderingActor` (A2), so it could not also extend an abstract `Infestation`. That reuse would be lost.
A reader would learn one type per side, instead of an interface plus a base class on each side.	`DeterrentItem` must extend the engine's `Item` to sit in the inventory, so it could not also extend an abstract `PestDeterrent`.
	A pest would inherit any default code in the parent whether it needed it or not, a Refused Bequest risk.

Design 2 (chosen): both are one-method interfaces: `Infestation.feedOn` and `PestDeterrent.deterAround`. A pest is an actor and an `Infestation` at once. A defence is an item and a `PestDeterrent` at once.

Pros	Cons
Each class keeps its own parent and adds the role on top. So `Pest` still reuses the A2 wandering code, and `DeterrentItem` stays an `Item` (DRY).	An interface holds no code. So the shared pest turn still needs an abstract class, `Pest` (Section B). A reader meets two types on each side. Acceptable because the two do different jobs. The interface says what others may ask. The abstract class shares how pests do it.
Callers see only the one method they call. A defence never sees how a pest moves, so a change to movement never reaches a defence (ISP).	

Finding the other side. Defences must tell a pest from the Farmer, a cow or a ladybug. Pests must tell a plant from bare ground.

Design 1: each defence checks `instanceof Pest`, and each pest checks for concrete crops before calling their methods.

Pros	Cons
The check is short and every Java reader knows it. The code says exactly which classes it acts on.	Each defence would name `Pest`, and each pest would name every crop and stage. The two sides would be tied to each other's classes (Connascence of Type).
	A new crop would edit all three pests, because each pest lists the crops it can eat (OCP, Shotgun Surgery).
	A creature that is not a `Pest` subclass, such as a cursed scarecrow, would be missed by every defence. A type check is the smell polymorphism is meant to replace.

Design 2: one class keeps a list of every pest class, and the defences ask it.

Pros	Cons
Every pest is named in one place, so a reader sees them all at once. The defences no longer name `Pest`.	Every new pest edits the list, so adding a pest is never just one new class (OCP).
	The list must be kept in step with the pest classes by hand. A pest left out is silently ignored by every defence (Connascence of Value between the list and the classes).

Design 3 (chosen): both sides ask for a capability. Defences use `PestPresent`, a `TileCondition` (the area filter from A2) that checks `actor.asCapability(Infestation.class)`. Pests use `getGroundAs(Regressible.class)`, from REQ1, and call `regress`.

Pros	Cons
Each side asks the other one question, through a capability. Defences never name `Pest`, and `game.pests` never imports `game.crops` (DIP).	A reader must know that a capability, not a class, decides who is a pest. Acceptable because the codebase uses the same lookup everywhere, for example `Regressible` and `Plantable`. A reader learns it once.
A new pest that implements `Infestation` is found by every defence. A new plant that implements `Regressible` is eaten by every pest. Neither needs an edit to the existing pests or defences (OCP).	A pest cannot treat one crop differently, such as a snail that prefers Strawberries. That would need a new method on `Regressible`, which every plant would then implement. Acceptable because no rule in the proposal treats crops differently.
The lookup is written once on each side, in `PestPresent` and in `Pest`, instead of once in every defence (DRY).	`PestPresent` is very small, close to a Lazy Class. Acceptable because three defences (trap, salt, ladybug) share it, so removing it would copy its check three times.

Foresight: a cursed scarecrow that is not a `Pest` could implement `Infestation`. Every trap, jar and salt bag would find it, with no change.

Connascence: between packages, only Connascence of Name (`feedOn`, `regress`) and of Type (`Infestation`, `Regressible`). No class depends on `SlimeSnail`, `LocustSwarm`, `BurrowingMole` or any crop. **Code smells:** avoided type checks (`instanceof`), Shotgun Surgery and Feature Envy (pests reaching into crop internals). Accepted: `PestPresent` is close to a Lazy Class.

B. One pest turn: feed or move, with movement as data

The pest turn.

Design 1: one `Pest` class with a `PestKind` enum and a `switch` in `feedOn`.

Pros	Cons
Every pest lives in one file, so a reader sees all the pest rules together (KISS).	Every new pest would mean editing the switch, and every other switch on the enum (OCP).
	One method would mix slime, area damage and cave-ins, so it changes for three reasons (SRP, the Switch Statements smell).
	Each case needs its own fields, such as the swarm's meals left. So the class carries fields most pests never use.

Design 2 (chosen): `Pest` is an abstract class (Template Method). Its turn is written once: if the pest can feed here, it returns an `InfestAction`. Otherwise its `SeekPlantBehaviour` moves it. A subclass gives only its landing offsets and its `feedOn`. One hook, `canFeedAt(Location)`, is true when a plant is underfoot. `LocustSwarm` overrides it, because a swarm also feeds on the plants next to it.

Pros	Cons
The rule "feed if you can, otherwise move" is written once. `Pest` extends the A2 `WanderingActor`, so its code is reused, not copied (DRY).	A reader must open two classes to see one pest's full turn. Subclasses also depend on two names in `Pest`: `feedOn` and `canFeedAt` (Connascence of Name). Acceptable because this is the price of writing the turn once, and each subclass depends on only those two names.
Adding a pest means writing one new class. `Pest`, `InfestAction` and the defences do not change (OCP).	A subclass could override `canFeedAt` badly and return true where `feedOn` finds nothing. Acceptable because `feedOn` then returns an empty report and changes nothing. The pest only wastes a turn.
The turn never asks which pest it is. The `canFeedAt` hook replaces `if (this instanceof LocustSwarm)`. So each pest works wherever `Pest` is expected (LSP).	The turn can only change through its two hooks. A pest that flees from a trap would need a new hook in `Pest`. Acceptable because no pest in the proposal flees. A new hook with a default would not change the existing pests.

How a pest moves. The proposal gives each pest its own way of moving. The swarm jumps like a knight, the snail crawls one tile and the mole burrows one or two.

Design 1: each pest overrides `move()` and runs its own nearest-plant search.

Pros	Cons
Each pest's movement sits in its own class, so a reader sees one pest's whole path in one file.	Three classes would each copy the search: find the nearest plant, pick a landing tile, or else wander (DRY).
	A bug fix would need three identical edits. Missing one would make one pest behave differently (Connascence of Algorithm).

Design 2: three movement classes, `KnightMove`, `CrawlMove` and `BurrowMove`.

Pros	Cons
Each movement has a name a reader can search for, such as `KnightMove`. A movement could later get its own logic.	The three classes would differ only in a table of numbers, so they add types without adding behaviour.
	The A2 marker called one class per variant "architectural bloat". Every new movement would need a new class, although it is only new numbers.

Design 3 (chosen): one `SeekPlantBehaviour` holds the search, and each pest gives only its landing offsets as an `int[][]`. The pattern is the only part that varies, so it is the only part each pest supplies.

Pros	Cons
The search is written once, in `SeekPlantBehaviour`. A bug fix reaches every pest at the same time (DRY).	The offsets are a raw `int[][]`, a mild Primitive Obsession smell. Acceptable because the array never leaves its pest class, and the behaviour copies it, so it cannot be changed later.
A new movement is a new offset table, with no new class and no edit to `SeekPlantBehaviour` (OCP).	Each row must be read as a `{dx, dy}` pair (Connascence of Meaning). Acceptable because the behaviour's constructor rejects rows that are not pairs, so a mistake fails at once.
No class per movement is needed, which avoids the "architectural bloat" the A2 marker named (KISS).	

The fallback is typed as the engine's `Behaviour`, not `WanderBehaviour` (DIP). A pest lands on a tile without walking through the tiles between. So a jumping pest can pass a wall or a salted tile. Two rules came from running the game. A mole that always moved two tiles could never reach a plant one tile away. So it now moves one or two. A knight jump rarely lands on a plant, so the swarm feeds from the tiles next to it. Unit tests cover both.

Why the offsets are written inside each pest. The A2 marker asked us to pass the Merchant its goods from outside. We did not do the same for the offsets. A Merchant that sells other goods is still a Merchant. But a snail

always crawls one tile. If it jumped like a knight, it would not be a snail any more. It would be a different pest, so we would write a new class for it.

Why `feedOn` is not a template. We considered a template on `Pest` where each pest gives only a per-tile step. But the three effects touch different tiles. The swarm hits the area around it. The mole hits its four straight neighbours. The snail hits the bare ground around it. They do different things to those tiles, and only the swarm changes itself (meals, splitting). A template would wrap three different effects without removing repeated code (KISS).

Foresight: an Aphid that hops three tiles and poisons the soil needs one new class. It holds a new offset table and a new `feedOn`. A grasshopper that leaps three tiles in a straight line is one new offset table. `SeekPlantBehaviour` stays the same.

Connascence: `InfestAction` knows a pest only through `Infestation` (Connascence of Name). Connascence of Algorithm between three copies of the search is avoided. The offsets carry Connascence of Meaning, but only `SeekPlantBehaviour` reads them, and its constructor rejects rows that are not pairs. **Code smells:** avoided Switch Statements, Duplicate Code and one class per variant ("architectural bloat"). Also avoided Speculative Generality: a template for effects that share nothing. Accepted: Primitive Obsession (`int[][]`), local and copied defensively. `InfestAction` is close to a Middle Man, because the engine needs an `Action` object for every turn.

C. One way to use every defence

Design 1: three actions, `SetTrapAction`, `OpenJarAction` and `ThrowSaltAction`, one for each defence.

Pros	Cons
Each action could have its own menu text and class name. A reader finds the trap's code by searching for "trap".	Each action would repeat "remove the item, act, report" (Duplicate Code). A change to how items are used would mean three edits (Connascence of Algorithm).
Each action would read alone, with no shared action class to learn first.	Every new defence would also need a new action beside its item. The two class trees would grow side by side (OCP).

Design 2 (chosen): `DeterrentItem` is an abstract item. While the Farmer carries it, it offers one `DeployDeterrentAction`, which calls `deterAround` and removes the item. The action holds a `PestDeterrent`, not a `PestTrap`. The hook `canDeployAt(Location)` hides the option where it cannot work. A trap is not offered on a plant, so the item is never wasted.

Pros	Cons
The steps "remove the item, act, report" are written once, in `DeployDeterrentAction`. A change to them is one edit (DRY).	The action keeps two fields for one object. It needs the `Item`, because the inventory only removes an `Item`. It needs the `PestDeterrent` to call it. Acceptable because both fixes are worse. A cast breaks type safety. A generic `<T extends Item & PestDeterrent>` adds a type parameter just to save one field.
The action only sees the defence as a `PestDeterrent`. A new defence that extends `DeterrentItem` needs no change to the action (OCP, DIP).	The two fields must hold the same object (Connascence of Identity). Acceptable because `DeterrentItem` passes `this` twice in one line, so no caller can mix them up.
The item says what it does. The action says how an item is used. Each changes for one reason (SRP).	

Foresight: a Scarecrow that scares pests away for ten turns is one `DeterrentItem` subclass plus one `TemporaryGround` subclass.

Connascence: the action's two fields must be the same object (Connascence of Identity), kept local in one line of `DeterrentItem`. **Code smells:** avoided Duplicate Code and a parallel hierarchy of one action per item. Accepted: two fields for one object, which is cheaper than a cast.

D. Temporary ground and the three defences

Slime, salt and the trap replace a tile's ground for a while and then put the old ground back. The trap and the salt are each an item and a ground. This looks like a Parallel Inheritance Hierarchy. But the engine keeps carried

things (`Item`) and map tiles (`Ground`) as separate types, and one object cannot be both. The item lives in the inventory. The ground lives on the map with its own timer.

Counting down.

Design 1: each temporary ground keeps its own counter and restores the old ground itself.

Pros	Cons
Each ground reads alone, with no abstract class to learn first (KISS).	The same countdown would sit in three classes (Duplicate Code). A rule like "rain shortens every temporary ground" would mean three edits (Shotgun Surgery).
	A ground could forget to restore the old ground, a bug the compiler cannot catch. The tile would keep the slime or salt forever.

Design 2 (chosen): `TemporaryGround` is abstract. Its `tick` is `final` and does three steps. First it runs `tickEffect`, which is empty by default. Then it stops if the effect already replaced this ground. Otherwise it counts down and puts the old ground back.

Pros	Cons
The order "effect, check, count down" is fixed in one place. A ground that extends `TemporaryGround` cannot skip putting the old ground back (Connascence of Execution).	`final` stops a subclass from changing the countdown itself, for example a ground that never expires. Acceptable because such a ground is not temporary, so it would extend `Ground` directly. Fixing the order is what stops the forgotten-restore bug.
`SlimeGround` and `SaltGround` are only constants, so a new temporary ground is a few lines (KISS, DRY).	

Package placement. `TemporaryGround` sits in `game.positions`. `SlimeGround` sits in `game.pests`, and `SaltGround` and `TrapGround` in `game.pestcontrol`. If all three were in `game.positions`, that package would import `game.pests`, which already imports `game.positions`: a cycle. Each ground sits beside the feature that creates it (Acyclic Dependencies Principle).

PestTrap and TrapGround (pull in). Each turn the trap pulls every pest within two tiles one step closer. Then it catches up to two pests on it or next to it. Grounds tick before actors move. In our first version the trap captured first. A pulled mole then walked away on its own turn, and the trap never caught it. Pulling first fixed this (Connascence of Timing, explained in the class comment and locked in by a test).

Printing the trap's report. A ground's `tick` returns nothing. So the trap cannot hand its report to the engine, as an action does.

Design 1: the trap keeps a `Display` in a field. It is either `static` and shared, or `private final` in each trap, as in `Plant`.

Pros	Cons
The `Display` is created once and is ready whenever the trap has news.	A `static` field is global state, which the A2 marker penalised. Every trap would depend on one hidden object.
The per-trap version matches REQ1's plants, so both features print the same way.	With a field in each trap, every trap holds a `Display` it rarely uses. The class diagram also gains an extra association.

Design 2: a `Display` is passed into the trap's constructor.

Pros	Cons
A test could pass a recording `Display` and check the printed text.	The `Display` would travel through `PestTrap` and `DeployDeterrentAction`, which have nothing to print, only to reach the ground (Tramp Data).
The trap would depend on an object it is given, not one it creates (DIP).	The engine's `Display` has nothing to swap in. And every test that builds a trap would have to build a `Display` too.

Design 3 (chosen): the trap creates a local `Display` only on a turn when it has news.

Pros	Cons
There is no static state and no field, so the trap has a dependency on `Display`, not an association.	Tests check the map, not the printed text. Acceptable because the map is what the player relies on, and the printed text only reports it.
The item and the action stay free of printing, so only the class with news creates a `Display` (KISS).	The trap creates its own `Display`, a small DIP gap. Acceptable because `Display` is the engine's console writer, with nothing to swap in. The trap only uses its `println` (Connascence of Name).

We could also make the trap an actor, so the engine prints its report. But a trap is not a creature, and this would change the design the TA approved.

LadybugJar and Ladybug (hunt). The jar releases up to three ladybugs around the Farmer. A ladybug eats a pest next to it, or wanders. It loses one hit point a turn, so it lives five turns. The existing `ConsciousActor` removes it at zero (DRY). The cost: lifespan and health are one number. If something healed a ladybug, it would live longer. Nothing in the game does.

SaltBarrier and SaltGround (push out). The bag salts the bare Soil and Dirt around the Farmer. Any pest on those tiles is burnt and pushed out. All of this happens once, inside `deterAround`. After that, `SaltGround` only refuses pests in `canActorEnter` (KISS).

Connascence: "effect, check, count down" is Connascence of Execution, kept inside one `final` method. Grounds ticking before actors move is Connascence of Timing, kept inside `TrapGround` and locked in by a test. Placing each ground beside its feature keeps package dependencies one way. **Code smells:** avoided Duplicate Code, Shotgun Surgery and a static `Display` (global state). Accepted: `final` stops a subclass from changing the countdown. The item-and-ground pairs also look like a Parallel Inheritance Hierarchy, for the engine reason above.

E. Visibility, `final`, `protected` and `static`

A2 feedback said package-private helpers (`CrowSensing`, `CrowNavigation`) can be good encapsulation, but the rationale must say so. Here is each choice in REQ3.

Choice	Where	Why	Cost and when we would change it
Every class is `public`	All 20 REQ3 types, including the small `PestPresent`, `InfestAction` and `EatPestAction`	Extensibility is a design goal, and other features already live in other packages. A defence in another package, for example a frog in `game.actors`, should reuse `PestPresent` and `EatPestAction` without moving them. Being `public` only makes reuse possible. Openness for extension comes from the interfaces and hooks, not from the modifier.	A wider public API. Only `Pest` uses `InfestAction`, so it could be package-private. We kept one rule for the whole feature instead. Then a reader never wonders whether a modifier was forgotten.
Every field is `private`	All classes	Implementation details stay inside (A2 feedback). Subclasses use hooks, not fields, so connascence on state stays inside each class.	`SlimeSnail`, `BurrowingMole` and `LocustSwarm` each keep their own `AreaFactory`, although `Pest` also receives it. A `protected` getter on `Pest` would remove three fields but expose a tool that `Pest` itself does not use.

Choice	Where	Why	Cost and when we would change it
Constants are `private static final`	e.g. `LocustSwarm.SPLIT_CHANCE_PERCENT`, `SlimeSnail.HIT_POINTS`	A fixed rule belongs to the class, not to one object. A name such as `SPLIT_CHANCE_PERCENT` explains the number, so there are no magic numbers. Java does not allow instance fields in `super(...)`, so names, characters and hit points must be static to be named.	A test cannot read them, so tests repeat the number. If a rule had to change at run time, it would become a constructor parameter instead.
No static methods, no `public` constants and no mutable static fields	Every REQ3 class	This is what the A2 marker penalised: static methods are "function thinking", and `public` constants leak details. A `private static final` constant does neither: it cannot change and is not seen outside the class.	Collaborators such as `RandomUtils` and `AreaFactory` must be passed in through constructors (Section F), which makes the constructors longer.
`protected` constructors	`Pest`, `DeterrentItem`, `TemporaryGround`	They are abstract, so only subclasses construct them, and `protected` says so in the code.	Classes in the same package could also call them. But Java does not let anyone create an abstract class, so nothing can misuse them.
`protected` hooks	`canFeedAt`, `canDeployAt`, `tickEffect`	Subclasses must override them, and callers must go through the template, not skip it.	Classes in the same package could also call them. No class does, and the templates that call them are fixed in the parent.
`protected final`	`TemporaryGround.revertAt`	A subclass may end its ground early (the trap after two captures) but cannot change how it ends.	A ground that wanted to end differently, such as leaving scorched earth behind, would need a new hook in `TemporaryGround`.
`final` method	`TemporaryGround.tick`	Fixes the order of the countdown (Section D).	A never-ending ground cannot reuse it and would extend `Ground` directly (Section D).
Classes are not `final`	e.g. `LadybugJar`, `SaltGround`	A larger jar, for example, could be a subclass. Not being `final` only makes a subclass possible. The design is open for extension because new classes plug into the abstractions (OCP).	A subclass could break a rule. Private fields and the final `tick` limit the damage.
`private` constructor	`LocustSwarm(int meals, ...)`	Only a split may create a smaller swarm. The public constructor always gives four meals, so a half-empty swarm cannot be built by mistake.	A future "hungry swarm" difficulty would need a new public constructor.
`private` helpers	e.g. `pullPests`, `splitAt`, `copyOf`	They are steps, not contracts. Making them `protected` would freeze their signatures for every subclass.	A subclass that wants a different pull must override `tickEffect`.

Choice	Where	Why	Cost and when we would change it
Non-final fields	`mealsLeft`, `capturesLeft`, `remainingTurns`	These values change every turn, so they cannot be `final`. Each is `private` and changed by one method only.	A reader must check that one method to see how the value changes over time.

Explicit modifiers, and why we keep `protected`. Two pieces of advice pull in different directions. The Week 9 lecture and our tutor suggest a narrow API. Keep a class package-private when no other package needs it. Avoid `protected`, because in Java it also opens a member to every class in the same package. But Java has no keyword for package-private. It means writing no modifier at all. The A2 marker penalised exactly that, as a "missing modifier".

Design 1: package-private classes and hooks (no modifier) wherever every user is in the same package.

Pros	Cons
This is the narrowest API, with fewer dependencies across packages, as the Week 9 lecture recommends.	A reader cannot tell a deliberate choice from a forgotten modifier, which the A2 marker penalised as "missing modifier".
	A pest or defence in another package could not extend `Pest` or `DeterrentItem`, which works against the extensibility goal.

Design 2: replace the template hooks with strategy objects, so no `protected` is needed.

Pros	Cons
No member would be `protected`, so no hook would be open to the rest of the package.	Each hook would need a new interface and class, only to avoid one modifier. That adds types but no behaviour (KISS).
	Every pest and defence would be built with extra strategy objects, so each constructor and `StackQueueValleyWorld` would grow.

Design 3 (chosen): explicit `public` classes and `protected` hooks.

Pros	Cons
Every class, and every member of a class, states its access. A reader never wonders whether a modifier was forgotten, which the A2 marker penalised. Interface methods are the one exception: Java makes them public anyway, so writing `public` there is redundant.	The API is wider than strictly needed. Acceptable because extensibility is a design goal, and a class in another package can extend `Pest` or `DeterrentItem`.
`protected` is only used for constructors of abstract classes and for hooks. No field is `protected`, so state stays private.	Classes in the same package could call a hook. Acceptable because none does. The methods that call the hooks are fixed in the parent, so a hook cannot skip the rules.

Connascence: `private` fields keep connascence on state inside each class. `protected` hooks create connascence only with subclasses, never with callers. **Code smells:** avoided public attributes, which make a Data Class that others manipulate, and missing modifiers. Accepted: a wider API than strictly needed.

F. Changes to the earlier design

REQ3 is new in A3. The only earlier class it changes is `StackQueueValleyWorld`, which places the pests and gives the Farmer the defences. Adding them made that class large, so we also split one class out of it (below).

Statics replaced by injection. REQ3's first version called the A2 static helpers `Area.within`, `Area.adjacentTo` and `RandomUtils.rollPercent`.

Design 1: keep calling the A2 static helpers.

Pros	Cons
No constructor changes. Every class can roll a chance or build an area in one call.	The A2 marker called static helpers "function thinking". Keeping them would repeat a mistake we already lost marks for.
	Every pest would depend on global classes. A test could only fix the random numbers by changing global state, so tests could affect each other.
	The team fixed this in the A2 classes. REQ3 would be the only feature still calling statics, so the code would follow two styles.

Design 2 (chosen): `StackQueueValleyWorld.initialise` creates one `RandomUtils` and one `AreaFactory`. Every REQ3 class that needs them gets them through its constructor.

Pros	Cons
A pest depends on the objects it is given, not on global classes, so tests can pass replacements. Both are concrete classes, so this is dependency injection, which supports DIP but does not complete it.	Every pest and defence constructor gained one or two parameters. `Pest` must pass them on to its behaviours (Tramp Data). Acceptable because these are the only two shared collaborators, and they are passed, not looked up globally.
Tests pass `FixedRandom` directly, and there is no hidden global state, so tests cannot affect each other.	The new parameters add Connascence of Position to the constructors. Acceptable because each constructor has at most two of them. They have different types, so the compiler catches a swap.

Splitting `StackQueueValleyWorld.` With every requirement placed in it, the class grew to 630 lines. Its set-up methods passed the same four values to each other. These were the map, a `Set<Location>` of taken tiles, `RandomUtils` and `AreaFactory`. One of them took eight parameters (a Data Clump and a Long Parameter List). The taken tiles were a bare `Set<Location>`, one for each map. The rule "pick a random free tile, then mark it taken" already sat in one method. But any set-up method could also change the set directly. And each call had to be given the right set for the right map. We asked the teaching team whether this class could be split, and they agreed, because it is not engine code.

Design 1: leave the class as it is.

Pros	Cons
Nothing changes, so no teammate's work conflicts with the split, and everything placed at the start sits in one file.	The Data Clump stays: every set-up method carries the same four values. A call could pass the farm's taken tiles with the mines map, and the compiler would not notice.
	The taken tiles stay a bare `Set<Location>` that any method can change. "Only change it through the placement methods" is a habit, not a rule the code enforces.

Design 2: split the class by area into four new classes: `FreeTiles`, `FarmSetup`, `MinesSetup` and `MapLinks`.

Pros	Cons
Each class would change for one reason. A change to the pests would not touch the mines or the ladders (SRP).	`FarmSetup`, `MinesSetup` and `MapLinks` would each be used once, from `initialise`. To see what is placed at the start, a reader would open four files instead of one.
The world itself would shrink to building the maps and calling each part in order.	Adding a pest would still mean editing the class that creates it, now `FarmSetup`. So OCP would not improve. The composition root only moves.

Design 3 (chosen): extract only `FreeTiles`, which holds one map, its taken tiles, `RandomUtils` and `AreaFactory`. It offers `reserve`, `reserveAround`, `placeGround`, `placeItem` and `placeActor`. It rejects a tile from another map. When no free tile is left, it throws an exception instead of searching forever. The farm and the mines each get their own `FreeTiles`, and the layout itself stays in `StackQueueValleyWorld`.

Pros	Cons
The taken tiles and the methods that change them now live in one object. Nothing outside can change the set (encapsulation).	`StackQueueValleyWorld` is still the largest class, at 471 lines. Acceptable because it is the composition root, where every feature is built and placed. It grows with each requirement by design.
Before, the map, its taken tiles and the randomness were always passed together. Now they are one object, so the set-up methods take fewer parameters. The longest went from eight to six (KISS, no Data Clump).	`StackQueueValleyWorld` still passes `RandomUtils` and `AreaFactory` to the methods that build actors (Tramp Data). Acceptable because those methods hand them on to the constructors, and only `FreeTiles` needed them for placing.
`FreeTiles` is called from 19 places, for both maps, so it is not a class created for one call.	When a map is full, setting up the world stops with an exception. Acceptable because both maps have far more tiles than things placed at random, so this only happens after a mistake. Failing at once shows the mistake instead of freezing the game.
`reserve` and `reserveAround` reject a tile from another map. So the farm's and the mines' taken tiles cannot be mixed up.	The check compares map objects by reference (Connascence of Identity). Acceptable because each `GameMap` exists once, and the comparison stays inside one private method.

The split did not change what the game places or where. The order of random draws is the same, and all tests pass. Three runs of the game showed every plant, ladder, circle and actor in place. Because `FreeTiles` is now its own class, it has its own tests. `FreeTilesTest` checks three things. A reserved or used tile is never picked. A tile from the other map is refused. A full map stops with an error.

Display character. `SaltBarrier` first used `k`, which clashed with the A2 `CauliflowerItem`. Earlier characters are fixed, so the new class changed to `f`.

Connascence: Connascence of Position in the constructors, with at most two such parameters. `FreeTiles` keeps the taken tiles and the methods that change them together. No other class depends on how the set is stored. **Code smells:** avoided global static state. Removed the Data Clump and Long Parameter List in `StackQueueValleyWorld`. Accepted: Tramp Data, because `RandomUtils` and `AreaFactory` still travel together through constructors, and a large composition root.

G. Known minor violations and why we accept them

Violation or cost	Why it is acceptable
`SlimeSnail` and `SaltBarrier` both ask "is this tile bare Soil or Dirt?" (two lines each). DRY is not fully met.	Rule of three: two short copies are cheaper to read than a new class. A third use would be the signal to extract a `BareFarmGround` condition.
`LadybugJar` catches `GameEngineException` and moves on. An empty catch is a smell.	The tile is checked with `canActorEnter` first. The catch only covers a tile taken in between. A comment says so.
`Ladybug`, `LadybugJar` and `SaltBarrier` use `getExits()`, while other classes use `AreaFactory`.	`getExits()` is the engine's own list of neighbours and needs no extra dependency. `AreaFactory` is used where we filter or search an area.
`TrapGround` moves actors, which is more than most grounds do.	Acting on the pests around it is the trap's whole job. Only the trap does this each turn, and the logic stays in one class.
Pests (except the swarm) never leave on their own.	On purpose: the defences are the counter to the pests. A meal limit could be added in `feedOn` only.

Summary

The final design uses the chosen design of every part. Interfaces are found by capability (A). The pest turn is a template, with movement as data (B). One action deploys every defence (C). One `final` countdown serves every temporary ground (D). Abstractions were added only where three pests or three defences already share code or must be told apart. The main cost is that one turn is split over two classes. A reader opens the pest and `Pest`, or the defence and `DeployDeterrentAction`. In return, the design passes a real OCP test. An Aphid

needs one new pest class. A Scarecrow needs one new `DeterrentItem` and one new `TemporaryGround`. Neither changes `Pest`, `SeekPlantBehaviour`, `DeployDeterrentAction`, `TemporaryGround` or any other defence. Only `StackQueueValleyWorld` places them.

Principle	Where it shows	Change it absorbs	Where it is only partly met, and why
SRP	Moving (`SeekPlantBehaviour`), the turn (`Pest`), finding pests (`PestPresent`), using an item (`DeployDeterrentAction`) and timing ground (`TemporaryGround`) each have one class.	A new landing rule edits `SeekPlantBehaviour` only. A new countdown rule edits `TemporaryGround` only.	`TrapGround` pulls, captures and prints. `LocustSwarm` feeds, splits and leaves. Each is one rule from the proposal, and splitting them would add classes used once.
OCP	Adding a pest, defence or temporary ground means writing one new class.	An Aphid, a Scarecrow or a Frost ground: no existing pest, defence or ground changes.	The new class must still be placed in `StackQueueValleyWorld`. That is the composition root, so this is expected.
LSP	Each of the three pests works wherever `Pest` or `Infestation` is expected, and each defence wherever `PestDeterrent` is. Hooks replace type checks. Each pest and defence has its own tests.	A new pest is fed, found and trapped with no new branch.	`canFeedAt` and `feedOn` must agree. If they do not, `feedOn` returns an empty report and changes nothing, so the pest only wastes a turn. `final tick` stops a ground from breaking the countdown.
ISP	`Infestation`, `PestDeterrent` and `Regressible` each have one method. Defences never see a pest's movement or health.	A pest that moves differently does not touch any defence.	`PestPresent` uses `Infestation` only to recognise a pest and never calls `feedOn`. This is not a marker interface: `InfestAction` calls `feedOn`.
DIP	`DeployDeterrentAction` depends on `PestDeterrent`, pests on `Regressible`, `SeekPlantBehaviour` on `Behaviour`. `RandomUtils` and `AreaFactory` are passed in through constructors, which lets tests replace them. Both are concrete classes, so this is dependency injection rather than full DIP.	A new crop, defence or fallback behaviour needs no change to the classes that use it.	`Pest` creates `SeekPlantBehaviour` and `WanderBehaviour`. `SlimeSnail` creates `SlimeGround`. `LadybugJar` creates `Ladybug`. Each has one outcome, so an injected factory would be a parameter nobody varies (YAGNI).

DRY and KISS. Each rule exists once: the pest turn, the landing search, finding a pest and the countdown. Each new abstraction is used by at least two classes. The one exception is `InfestAction`, which the engine needs as an `Action` object. **Connascence.** The connascence we found between packages is of Name and Type. Stronger forms (Timing, Identity, Execution) stay inside one class each, with a comment and a test.

Testing. Each pest, defence and temporary ground is tested with normal, edge and invalid cases. `FixedRandom` makes them repeatable, and `mvn test` runs them all.

Yuanshu Liu 36226173 – REQ 4: Enchanted Tools

The Farmer can enchant a Hoe, a Watering Can or a Pickaxe. Each of these tools is now `Enchantable`. The Enchantments are Blaze, Bounty and Shatter, each a new `Enchantment` class. The next use of the tool fires its own spread and then the Enchantment on every tile the spread touched. The Enchantment is then spent. We had three design goals. Any tool can carry any Enchantment, and the actions never name a tool. The store-fire-clear bookkeeping is written once. Existing effects are reused instead of building new ones. Abstractions were added only where three tools or two actions already share code (KISS, YAGNI).

A. Finding and firing the tool that was used, without `instanceof`

After tilling, watering or breaking, the action must fire the Enchantment of the exact tool it used. This needs a capability, a shared lookup and a way to recognise that one tool.

A capability, not a type check. Two actions use tools: `CapabilityToolAction` (tilling and watering) and `BreakAction`. Each must fire the Enchantment of the tool it just used.

Design 1: check the tool's class, for example `if (tool instanceof Hoe)`, and read its enchantment directly.

Pros	Cons
The check is short and needs no new interface or helper (KISS).	Every new enchantable tool adds a branch to two actions, the Switch Statements smell spread over two classes (OCP).
A reader sees in the action itself which tools can carry an Enchantment.	Both actions would name every tool class (Connascence of Type). Renaming or adding a tool would mean editing both actions (Shotgun Surgery).
	The tests of both actions would have to cover every tool. A forgotten case would only fail while playing.

Design 2 (chosen): a capability interface, `Enchantable`, with `enchant(Enchantment)` and `triggerEnchantment(Actor, Location)`.

Pros	Cons
Both actions reach the tool's Enchantment only through `Enchantable`, so they compile without knowing any tool class (DIP).	It is one more interface to read. Acceptable because it follows the existing `Tillable` and `Waterable` pattern, so a reader already knows the shape.
A new enchantable tool that implements `Enchantable` needs no change to either action (OCP).	The actions still need a way to find the tool's `Enchantable` view, because the engine hands them a narrower type. Acceptable because the shared lookup below solves this once for both actions.
The tool, which owns the Enchantment, decides whether it has anything to fire (encapsulation).	A new tool must provide both methods of `Enchantable`, even if it only wants to fire. Acceptable because extending `AbstractEnchantableTool` already provides `enchant()`, so a tool writes only `triggerEnchantment()`.

One shared lookup. After their own effect, both actions need the same step. They find the tool just used among the actor's `Enchantable` items, and fire it.

Design 1: write the lookup loop inside each action.

Pros	Cons
There is no new class. The lookup sits next to the code that needs it, so each action reads alone.	The same loop would sit in two unrelated actions (Duplicate Code). A third action that uses a tool would copy it again.
	If the lookup changed, both copies would have to change together (Connascence of Algorithm).

Design 2 (chosen): `EnchantmentTrigger.fire(Actor, Object usedTool, Location)`, used by both actions. In the first version only `CapabilityToolAction` needed the lookup. When `BreakAction` needed the same loop, we

moved it into one class. So the class was made for a real second user, not "just in case".

Pros	Cons
One place knows how to go from a tool in hand to its `Enchantable` view (SRP).	The `usedTool` parameter is typed `Object`, so a wrong argument fails silently instead of at compile time. Acceptable because the two callers hold tools of unrelated types (Section F). A narrower type would need a new interface just for this.
A future action that uses a tool, such as a scything action, reuses it with no new code (OCP).	A new `EnchantmentTrigger` object is created on each use. Acceptable because it holds no state, so creating it only reads oddly and costs nothing.
The loop exists once, so a fix to the lookup reaches both actions (DRY).	

Recognising the exact tool. The engine's `Inventory.getItemsAs(Class)` returns views narrowed to one capability. A tool held as a `TillingTool` cannot be asked for a second capability, and the engine cannot be changed. So `EnchantmentTrigger` asks for a second view, `getItemsAs(Enchantable.class)`. Then it looks for the entry that is the same object as the tool in hand.

Design 1: `instanceof Enchantable` plus a cast.

Pros	Cons
It is short and familiar to Java readers, and needs no second inventory view.	It asks "what class is this?", which is the wrong question. It also brings back the Connascence of Type that the interface removed.
	A type check plus a cast is the smell polymorphism should replace. If the check is ever wrong, the cast fails at run time, with an error far from its cause.

Design 2: compare with `equals`.

Pros	Cons
It is familiar, reads like an ordinary value comparison, and would keep working if tools later defined their own equality.	Tools do not override `equals`, so it would quietly mean `==` anyway. A reader would wrongly expect two equal-looking tools to match (Connascence of Meaning). A later `equals` override, added for another reason, would silently change which tool fires.

Design 3 (chosen): compare references with `==` (`enchantable == usedTool`).

Pros	Cons
It asks the real question, "is this the exact tool used this turn?", with no cast and no type check.	It is Connascence of Identity, the strongest kind. Acceptable because both references are found and compared inside one method, and never stored. The rule cannot leak to other classes.
Two identical Hoes are told apart, because the comparison is by reference, so only the tool actually used matches.	

Foresight: an enchantable Scythe implements `Enchantable` through the base class in Section B, and the actions stay the same.

Connascence: the actions only depend on the type `Enchantable`, not on three tool classes (Connascence of Type). They also depend on the name `fire` (Connascence of Name). The one Connascence of Identity (`==`) stays inside one method. **Code smells:** avoided type checks, casts, Switch Statements on the tool's class and Duplicate Code. Accepted: a weakly typed `Object` parameter.

B. Sharing enchanting: one base class and one slot

All three tools store, offer, fire and clear Enchantments the same way. Only their spread differs.

One base class. All three tools must do the same three things. They store the Enchantments they offer, offer one `EnchantAction` for each, and hold the one attached. None of this depends on the tool.

Design 1: each tool implements `Enchantable` on its own.

Pros	Cons
There is no new abstract class, so each tool reads alone, top to bottom, without a parent to open (KISS).	The same storage and menu code would be written three times (Duplicate Code). A fourth tool would copy it again.
	A change to how `EnchantAction`s are offered would need three edits (Divergent Change, Connascence of Algorithm).

Design 2 (chosen): `AbstractEnchantableTool extends Item implements Enchantable`. It owns the offered Enchantments, the attached one (through `EnchantmentSlot`, below), `enchant()` and the menu. `Hoe`, `WateringCan` and `Pickaxe` extend it and write only `triggerEnchantment()`. An interface default method cannot hold a field, so sharing the stored Enchantment needs a class. This is a language limit, not a style choice.

Pros	Cons
Storage and the menu are written once, so a fix reaches all three tools (DRY).	A tool that should offer only some Enchantments would have to override `allowableActions()`. Acceptable because no tool needs a partial menu today (YAGNI).
Each tool changes only when its own spread changes, because the bookkeeping now lives in the parent (SRP).	Subclasses reach the slot through the `protected` accessor `enchantmentSlot()`, a Connascence of Name that a rename would break. Acceptable because the accessor is the only member subclasses depend on, never a field.
Each of the three tools works in `EnchantAction`, because all of them inherit the same `enchant()` (LSP).	

One slot that stores, fires and clears. Firing is the same for every tool. If an Enchantment is attached, apply it to the affected tiles, then clear it. What differs is which tiles the tool's own spread touches.

Design 1: write the store, fire and clear steps inside each tool's `triggerEnchantment()`.

Pros	Cons
There is one class fewer, and everything about one tool's turn sits in one method, easy to read once.	One method would mix the bookkeeping with the tool's spread, so it changes for two reasons (SRP). Each tool's tests would also cover the bookkeeping again.
	The steps "check, apply, clear" would be written three times. Forgetting "clear" in one copy would let an Enchantment fire forever (Connascence of Execution).

Design 2 (chosen): `EnchantmentSlot`, a small helper. It holds at most one Enchantment. It offers `enchant()`, `isEmpty()` and `fire(...)`, which applies the effect and then clears the slot.

Pros	Cons
Each tool only builds its list of tiles and hands it to the slot. So the tool holds only its own spread (SRP).	Tracing a turn needs one more step: tool, then slot, then Enchantment. The slot is close to a Middle Man. Acceptable because the extra step is what keeps the one-use rule in one place.
"Apply, then clear" is written once, so a tool that fires through the slot cannot skip the clear step (DRY).	The slot holds `null` while empty. Acceptable because the `null` stays private and callers only see `isEmpty()`.
Other small helpers in the code work the same way, such as `StatisticAdjuster`. So the pattern is familiar.	

Foresight: a fourth enchantable tool writes only its `triggerEnchantment()`.

Connascence: subclasses depend on the parent only by Name (`enchantmentSlot()`), never on its fields. "apply, then clear" is Connascence of Execution, kept inside one method. **Code smells:** avoided Duplicate Code and Divergent Change. Accepted: one extra level of indirection (tool, slot, Enchantment).

C. Enchantment effects: reuse, and one contract

Each Enchantment reuses an existing effect, and all of them report in the same way.

Enchantment	Reuses	Instead of
`BlazeEnchantment`	The existing `Fire` item	A new burning mechanism
`ShatterEnchantment`	The existing `StunnedStatus`	A new stun status
`BountyEnchantment`	REQ1's `Plant.forceGrowth()`, which tries the plant's own abilities at once, without ageing it or printing	A growth loop of its own

Reusing existing classes.

Design 1: each Enchantment writes its own fire, stun and growth logic.

Pros	Cons
Each Enchantment class stands alone and can be read without opening other packages.	There would be a second burning rule and a second growth loop. Both must be kept in step with the originals (Duplicate Code, Connascence of Algorithm).
	A frost pause or a new growth rule would have to be added in two places. Missing one would make magic growth differ from natural growth.

Design 2: Bounty calls `Plant.attemptGrowth()`, the method the plant uses on its own turn (our first version).

Pros	Cons
Bounty needs no new method in `Plant`. It uses exactly the rules of a normal turn.	`attemptGrowth` checks each ability's interval and chance first. On most turns a plant is "not due", so the magic silently did nothing.
	The player spent an Enchantment and saw no growth, which breaks the proposal's promise that Bounty forces growth.

Design 3 (chosen): reuse `Fire` and `StunnedStatus`, and call `Plant.forceGrowth()` for Bounty. `forceGrowth` tries the same abilities, but skips their interval and chance (REQ1, Section B).

Pros	Cons
Each effect already exists and is already tested. There is no second burning rule and no second growth loop in `game.enchantments` (DRY, KISS).	`game.enchantments` depends on `game.items`, `game.status` and `game.plants` (Connascence of Name across packages). Acceptable because these are stable public methods, and there is no package cycle.
Bounty now grows a plant on any turn, as the proposal says. Three tests check this on turns a plant is not due: a seedling at ages 2 and 4 grows, and a strawberry at age 2 drops fruit.	`Plant` gained a second growth method, so its short loop is written twice. Acceptable because the copies differ in one call and sit side by side in one class (REQ1, Section G).
`forceGrowth` checks REQ5's frost pause too, so Bounty does not grow a paused plant. A test checks this.	

The empty-string contract. `Enchantable.triggerEnchantment`, `EnchantmentSlot.fire` and `Enchantment.applyTo` must say when nothing fired.

Design 1: return `null` for "nothing happened".

Pros	Cons
It is common in Java and cheap to write. A caller can test for it with one `if`.	Every caller must remember what `null` means (Connascence of Meaning), and a forgotten check crashes the turn. Scattered `null` checks are a known smell.
	REQ1's abilities already use `""` for the same meaning, so two features would say "nothing" in two different ways.

Design 2: throw an exception when nothing fired.

Pros	Cons
No caller can ignore it. The exception can also say why nothing fired.	An empty slot is normal, not an error. Yet every caller would need a `try/catch` for it, repeated in each caller (Connascence of Execution).
	Exceptions would be used for normal events. Real errors would be harder to spot among them.

Design 3 (chosen): return `""`, the same rule REQ1's abilities use.

Pros	Cons
Every caller simply joins the strings, with no `null` check and no `try/catch` around the call (KISS).	A reader must know that `""` means "nothing happened" (Connascence of Meaning). Acceptable because the Javadoc of each method says so. It is also one rule for two features, not a different rule per method.
All implementations keep one contract, shared with REQ1's abilities, so any Enchantment can replace another (LSP).	

Every message still names a location. With no affected tiles, each Enchantment reports "near" the tool's tile. Both actions start their message with "At" and the location.

Connascence: Connascence of Name on three stable public parts of other packages: `Fire`, `StunnedStatus` and `Plant.forceGrowth`. There is also one Connascence of Meaning: `""` means "nothing happened". It is documented, and REQ1 uses the same rule. **Code smells:** avoided Duplicate Code, parallel mechanisms for the same effect and scattered `null` checks.

D. Visibility, `final`, `protected` and `static`

Choice	Where	Why	Cost
Every class is `public`	All REQ4 classes	Another package can reuse them, for example a new action that uses `EnchantmentTrigger`. Being `public` only makes reuse possible. Openness for extension comes from the interfaces and hooks, not from the modifier.	A wider public API, so more classes could depend on them (more connascence across packages).
Every field is `private`	All classes	Subclasses use `enchantmentSlot()`, not the field, so no Data Class is manipulated from outside.	A subclass that needs more of the parent's state must ask for a new accessor.
The offered list is copied	`AbstractEnchantableTool` keeps `List.copyOf(offeredEnchantments)`	A `private` field alone does not stop a caller changing the list it passed in. With the copy, adding to or removing from that list later does not change a tool's menu (no mutable aliasing).	One small copy per tool, made once at construction.
`protected` accessor	`AbstractEnchantableTool.enchantmentSlot()`	Only the tools need the slot to fire it. Callers use `Enchantable`.	Classes in `game.items.tools` could also call it. None does.

Choice	Where	Why	Cost
`protected` constructor	`AbstractEnchantableTool`	It is abstract, so only tools construct it, and `protected` says so.	Same-package classes could call it, but Java forbids creating an abstract class anyway.
`final` classes	`EnchantmentSlot`, `EnchantmentTrigger`	Small helpers used by composition, not by inheritance. `EnchantmentSlot` promises to hold at most one Enchantment and to clear it after firing. A subclass could override `enchant`, `isEmpty` or `fire` and break that promise. Making only `fire` final would not be enough. They are the only `final` classes in the game. Every other class is open for extension.	Neither class can be extended. A tool that needs two Enchantments would hold two slots, or a new class that holds several. This extends the behaviour by composition and keeps each slot's one-Enchantment promise. Limiting which Enchantments a tool offers needs no subclass either. Each tool gets its own list when it is built.
Constants are `private static final`	e.g. `CHAIN_BREAK_CHANCE_PERCENT`, `STUN_DURATION_TURNS`	A name explains each number, avoiding magic numbers (Connascence of Meaning). No public constants (A2 feedback).	Tests repeat the numbers.
No static methods	Every REQ4 class	A2 feedback penalised static methods as "function thinking".	`EnchantmentTrigger` is created with `new` in each action instead of being called statically.

Connascence: `private` fields and the copied list keep connascence on state inside each class. **Code smells:** avoided public attributes, mutable aliasing, magic numbers and missing modifiers.

E. Changes to the earlier design

Change	Why	Cost
`Hoe`, `WateringCan` and `Pickaxe` extend `AbstractEnchantableTool` instead of `Item`.	They are the three tools changed to carry an Enchantment (Section B).	An unenchanted tool behaves as before. Its menu gains the enchant options.
`CapabilityToolAction` and `BreakAction` call `EnchantmentTrigger` after their own effect.	So every tool use can fire its Enchantment (Section A).	One more call in each action.
`TillingTool` and `WateringTool` declare `canTill(Tillable)` and `canWater(Waterable)`.	A2 feedback: empty marker interfaces are not good OO design.	Every tool returns `true` today.
`Enchantment` and `EnchantAction` live in `game.items.tools`.	Keeping them there removes a package cycle between `game.items.tools` and `game.enchantments`.	The interface sits beside the tools, not beside its implementations.
The tools receive their Enchantments through the constructor.	No hard-coded list inside the tools. `StackQueueValleyWorld` decides what is offered.	One more constructor parameter.

F. Known minor violations and why we accept them

Violation or cost	Why it is acceptable
`EnchantmentTrigger.fire` takes `Object usedTool`, so a wrong argument fails silently (no match, `""`) instead of at compile time.	The two callers hold tools of different types: `T extends HasStaminaCost` and `BreakingTool`. `HasStaminaCost` does not say the tool is an `Item`. A narrower type would need a new shared interface or a new generic bound, just to tighten one parameter (YAGNI).
The three `triggerEnchantment()` bodies share a shape: check the slot, build a tile list, fire, join the messages.	Their spreads differ in kind: every adjacent tile, one random tile, or one random tile behind a chance roll. What they share, the slot logic, is already in `EnchantmentSlot`. With three different shapes, a common template would be a guess (rule of three).
`game.enchantments` depends on three other packages (Section C).	Reuse is the point. There is no cycle.
A Hoe or Pickaxe rarely carries Bounty to a plant, because tilled or broken tiles hold no plant.	The proposal says Bounty forces growth on any plant on an affected tile. With the Watering Can, it makes a watered crop produce or spread again. `WateringCanTest` checks this through the real tool.

Summary

The final design uses the chosen design of every part. The used tool is found through the `Enchantable` capability, by reference and not by class (A). One base class and one slot hold the shared bookkeeping (B). Enchantments reuse existing effects under one `""` contract (C). Each abstraction removes code that three tools or two actions already shared. The main cost is extra steps. A turn passes from the action through `EnchantmentTrigger`, the tool and the slot to the Enchantment. In return, the design passes a real OCP test. An enchantable Scythe needs one new tool class. A Frost Enchantment needs one new `Enchantment` class. Neither changes either action, `AbstractEnchantableTool`, `EnchantmentSlot` or any other Enchantment. Only `StackQueueValleyWorld` offers them.

Principle	Where it shows	Change it absorbs	Where it is only partly met, and why
SRP	Each job has one class. `AbstractEnchantableTool` offers. `EnchantmentSlot` stores and fires. `EnchantmentTrigger` finds the used tool. Each tool has its spread, and each Enchantment its effect.	A new firing rule edits only `EnchantmentSlot`.	Each tool's `triggerEnchantment()` both spreads and joins the messages. Splitting them would add a class per tool.
OCP	Adding a tool or an Enchantment means writing one new class.	A Scythe or a Frost Enchantment: no existing tool, action or Enchantment changes.	New Enchantments are listed in `StackQueueValleyWorld`, the composition root.
LSP	Each of the three tools works in both actions, and each of the three Enchantments in any slot. Each tool and each Enchantment has its own test class.	A new tool fires with no new branch.	All Enchantments must keep the `""` contract. The Javadoc states it.
ISP	`Enchantable` has two methods and `Enchantment` one. The actions see only `Enchantable`.	A new Enchantment implements one method and sees nothing of the tools.	`Enchantable` joins enchanting and firing. Every enchantable tool needs both, so splitting would add a type nobody uses alone.
DIP	The actions depend on `Enchantable`, the tools on `Enchantment`. `RandomUtils` and `AreaFactory` are passed in through constructors, which lets tests replace them. Both are concrete classes, so this is dependency injection rather than full DIP.	A new Enchantment needs no change to the tools.	`EnchantmentTrigger` is created with `new` in each action. It has no state and one possible behaviour.

DRY and KISS. The menu, the slot and the lookup are each written once. The Enchantments reuse `Fire`, `StunnedStatus` and `Plant.forceGrowth`. **Connascence.** The connascence we found between packages is of Name and Type. The one Connascence of Identity (`==`) stays inside one method.

Testing. Each tool, the slot and each Enchantment has its own test class, such as `HoeTest` and `EnchantmentSlotTest`. They cover each tool's spread, with and without an Enchantment. They also cover Enchantments with and without affected tiles. Bounty is tested on turns a plant is not due to grow, and on a paused plant it cannot grow. An Enchantment is spent after one use. `AbstractEnchantableToolTest` checks that changing the list given to a tool does not change its menu. `FixedRandom` makes them repeatable, and `mvn test` runs them all.

Jingyu Zhang 37033956, Yuanshu Liu 36226173, Poyee Tan 36641162 – REQ 5: The Wanderer (API, HD)

The Wanderer's tile is turned into a real latitude and longitude, and OpenWeatherMap's weather there decides what it does. Every 5 turns, today's weather triggers a `WeatherReaction` (storm shelter, rain dance or campfire). A Farmer next to it can ask for tomorrow's forecast, which brings a `ForecastOmen` (frost, sowing or drought). We had four design goals. The game does not depend on the weather API. The game keeps running when no weather can be read. Each part of a request has one class. Adding a weather effect means writing one new class. We only added an abstraction where it was needed. Either the proposal names six effects, or the network must stay out of tests (KISS, YAGNI).

A. The game asks an interface, and "no reading" is a normal answer

The Wanderer never talks to OpenWeatherMap directly, and a failed request must not stop the game.

Keeping the game independent of the weather API.

Design 1: `Wanderer` and `AskForecastAction` build the URL, send the request and read the JSON themselves.

Pros	Cons
There is one interface and one class fewer to read, and the whole weather path sits in one place (KISS).	Game classes would depend on `HttpClient` and on OpenWeatherMap's JSON shape. Another company controls that shape, and it can change at any time (DIP, Connascence of Algorithm).
	Every test of the Wanderer would need the network or a fragile HTTP mock, so tests would not be repeatable.
	Changing the weather provider would mean editing `Wanderer` and `AskForecastAction`. Neither has anything to do with HTTP (Shotgun Surgery).

Design 2 (chosen): a `WeatherSource` interface with `currentWeatherAt(lat, lon)` and `tomorrowWeatherAt(lat, lon)`, both returning `Optional<WeatherReading>`. `OpenWeatherClient` is the real one and is injected through the constructor. Tests inject `FixedWeatherSource`.

Pros	Cons
Game classes depend on an interface the game owns, not on the HTTP library (DIP). A change in the library stays inside `game.weather`. It does not reach the game classes, as long as the four values stay the same.	There is one more interface and one more class to learn before the feature makes sense. Acceptable because they are what keep the network out of the game and its tests.
`FixedWeatherSource` replaces the real client, so no REQ5 test uses the network (LSP).	The interface only carries the four values the game uses, so extra data needs a change to `WeatherReading`. Acceptable because a small interface is what lets any provider fit it. The proposal's rules only use these four values.
Another provider, or a cached one, needs no change to `Wanderer`, `AskForecastAction` or `WeatherInterpreter` (OCP).	

The package `game.weather` imports nothing from the rest of the game, so it could be reused by any feature.
Foresight: a weather source that replays saved readings for a demo needs one new class.

When the sky cannot be read. With no key, no network or a bad reply, there is no reading.

Design 1: throw a checked exception.

Pros	Cons
It is normal Java, and the compiler makes every caller handle the failure. The exception can also say why the reading failed.	No reading is a normal case, not an error. Yet every caller would need its own `try/catch` for it (Duplicate Code).

Design 2: return `null`.

Pros	Cons
It is short, needs no wrapper type, and every Java reader already knows the pattern (KISS).	A forgotten check crashes the game, and scattered `null` checks are a known smell.
	The rule "`null` means no reading" is invisible to the compiler and lives only in comments (Connascence of Meaning).

Design 3 (chosen): return `Optional<WeatherReading>`.

Pros	Cons
"No reading" is part of the return type, so every caller sees it. Every source keeps the same contract (LSP).	A caller that calls `get()` without checking still fails. So the risk is smaller, but not gone. Acceptable because there are only two callers, and both check `isEmpty()` first.
The game handles it in one place per caller with `isEmpty()`, with no exception handling.	`Optional` invites functional chains, which the team avoids. Acceptable because we use it only with `isEmpty()` and `get()`, with no chained calls.

When there is no reading, the Wanderer just wanders. `UnreadableSkyAction` reports "The sky is unreadable, and the Wanderer only wanders." `AskForecastAction` reports that tomorrow's weather is unreadable. A missing reading does not stop the game, and the player is told what happened.

Connascence: Connascence of Algorithm with OpenWeatherMap's JSON layout stays inside `game.weather`. The rest of the game shares only Connascence of Type on `WeatherSource`. Returning `Optional` avoids the Connascence of Meaning a `null` result would need. **Code smells:** avoided Shotgun Surgery (changing provider would otherwise edit game classes), network calls in tests and scattered `null` checks.

B. The API layer: place, key and reply each have one class

Behind `WeatherSource`, three small classes each own one part of a request. One says where to ask. One finds the key. One reads the reply.

Turning the Wanderer's tile into a real place. The brief says the request cannot be a fixed URL. The game state must change the query.

Design 1: always ask about one fixed place, such as Melbourne.

Pros	Cons
It is easy to write and needs no extra class. The same answer for every tile also makes testing easy (KISS).	It breaks the brief's rule against a fixed URL. The game state would not change the query.
	The coordinates would be magic numbers inside a URL string, with nothing to say which number is which.

Design 2 (chosen): `MapGeography` turns a tile's (x, y) into a latitude and longitude. The top-left tile is Melbourne (-37.81, 144.96), and each tile is 2 degrees. `Wanderer` and `AskForecastAction` ask it about the tile the Wanderer stands on, so every move changes the query. Latitude stops at the poles and longitude wraps past 180, so any tile gives a valid place.

Pros	Cons
The query is driven by the game state, as the brief requires.	It is one more small class, and its constructor takes three `double`s in a fixed order (Connascence of Position). Acceptable because the constructor is called once, in `StackQueueValleyWorld`.
One class knows how map tiles match the real world (SRP). Moving the farm only changes its constructor arguments.	Two degrees per tile puts many tiles over the sea, so many readings come from ocean weather. Acceptable because the scale is one constant to change, and ocean weather still varies.
`latitudeOf` and `longitudeOf` name the two numbers, so callers cannot swap them.	

Keeping the API key out of the code.

Design 1: `OpenWeatherClient` calls `System.getenv("OPENWEATHER_API_KEY")` itself.

Pros	Cons
There is one class fewer, and reading the key is one line (KISS).	The client would also decide where secrets live. Moving the key would mean editing the network class (SRP, Divergent Change).
	A test could not control the key without changing the real environment, so tests would depend on external state.

Design 2 (chosen): `ApiKeyLoader` takes the key's name, an environment map and a `.env` path. It checks the environment first, then a `NAME=value` line in `.env`, and returns `Optional<String>`. The client receives a loader through its constructor.

Pros	Cons
The loader finds the key and the client calls the API, so each changes for one reason (SRP).	It is one more class and one more constructor parameter for the client. Acceptable because keeping a secret out of the code is worth one small class.
A test passes a plain map and a temporary file, with no real environment.	The key's name must match in the loader, the `.env` file and the README (Connascence of Name across files). Acceptable because `StackQueueValleyWorld` holds the name once.
The key is never in the source, and `.env` is in `.gitignore`, so it cannot be committed by mistake (security).	

Reading the reply.

Design 1: read the JSON inside `OpenWeatherClient`.

Pros	Cons
There is one class fewer, and the request and its reply sit together (KISS).	The client would both talk to the network and know the JSON layout. Two different kinds of change would edit the same class (SRP, Divergent Change).
	The JSON could only be tested through a network call, or by overriding the client, instead of with saved text.

Design 2 (chosen): `OpenWeatherParser` turns a reply into a `WeatherReading`. The weather condition and the temperature are required. Wind and rain default to 0, because OpenWeatherMap leaves out `rain` on dry days. For the forecast, it reads the last of nine 3-hour steps, about 24 hours ahead. It divides `rain.3h` by 3 to get rain per hour. A reply missing a required field gives no reading. `WeatherReading` is a small value object with `private final` fields. Its constructor rejects a blank condition, a value that is not a finite number, and negative wind or rain.

Pros	Cons
Only the parser knows the JSON layout (SRP). It is tested with saved JSON text, with no network.	Gson is an extra library, and the parser still knows the field names (Connascence of Algorithm with the API format). Acceptable because Gson comes through Maven, and the field names are confined to one class.
A bad reading cannot be created at all. So the reactions do not repeat these checks.	A reply with finite but odd values, such as 500°C, is still accepted. Acceptable because each effect has a time limit (below). An odd value only makes an effect longer or shorter, within that limit. The tests cover extreme values.
`WeatherReading` is not a passive Data Class, because it validates and describes itself.	The checks live in the constructor, so a test cannot build a broken reading to see how an effect reacts. Acceptable because a broken value takes the real path instead. The parser turns it into "no reading", and the tests cover that path.

Bad values. The parser catches a rejected reading and turns it into "no reading". The game then takes the normal unreadable-sky path. `StormShelterReaction` and `FrostOmen` turn the wind or the cold into extra turns. Each adds at most 100 turns, far more than any real weather needs. Without the cap, a huge value would overflow `int` and make a negative duration. Tests cover infinite values and extreme finite ones.

Connascence: `latitudeOf` and `longitudeOf` name the two numbers, reducing Connascence of Position, though the three-`double` constructor still has it. The key's name is written in one place (Connascence of Name). Only the parser depends on the API's format (Connascence of Algorithm). **Code smells:** avoided a static URL, magic numbers, a secret in the source code and Divergent Change in the client. Also avoided Primitive Obsession (four loose numbers) and a passive Data Class, because `WeatherReading` validates and describes itself.

C. From a reading to an effect: one interpreter, two interfaces

A reading must become a reaction today or an omen tomorrow, without the Wanderer knowing the rules.

Choosing which effect a reading triggers. The proposal fixes six rules. One is "wind of 10 m/s or more builds a shelter".

Design 1: an if-chain inside `Wanderer.playConsciousTurn` that also performs each effect.

Pros	Cons
There is no extra class, and the rules sit where they are used (KISS).	The actor would also hold every weather rule (SRP), and a new reaction would edit `Wanderer` (OCP).
	A reader would have to guess what the bare numbers mean (Connascence of Meaning). The forecast action would also need a second copy of the rules.

Design 2: a configurable rule engine: a list of (condition, effect) pairs, perhaps read from a file.

Pros	Cons
Rules could change without recompiling, and a new rule would edit no class (OCP).	The six rules are fixed by the proposal, so the flexibility would never be used (YAGNI).
	A general list is harder to read than a few `if`s. It solves a problem we do not have (Speculative Generality, KISS).
	Conditions read from a file would lose compile-time checking, so a typo would only show up while playing.

Design 3 (chosen): `WeatherInterpreter` with `reactionFor(reading)` and `omenFor(forecast)`. It is the only class that creates the six concrete classes, so it works as a simple factory.

Pros	Cons
`Wanderer` sees only `WeatherReaction`, and the rules have one home (SRP, DIP).	A new rule adds one `if` here, so OCP is only partly met. Acceptable because this is the one place that creates effects. Everything else stays closed. A few named `if`s are easier to check than a rule engine.
Each threshold is a named constant, written only here. A reader can check every rule against the proposal in one class (DRY).	This one class names all six concrete classes (Connascence of Name with each). Acceptable because it is the only class that creates them, so no other class names them.
Today's and tomorrow's rules share helpers such as `isRainy`, so a condition is written once.	

Thresholds passed in, not repeated. `CampfireReaction` and `FrostOmen` grow with the cold. They measure how far the temperature is below their threshold.

Design 1: each of the two classes keeps its own copy of the threshold (our first version).

Pros	Cons
Each class reads alone, with the number it needs written beside the code that uses it.	10°C and 2°C would be written twice (Connascence of Value across classes).
The two effect constructors stay one parameter shorter, so creating an effect needs one value less.	Changing a threshold in `WeatherInterpreter` would leave the effect sized against the old number, and nothing would warn us.

Design 2 (chosen): `WeatherInterpreter` passes the threshold into their constructors.

Pros	Cons
Each number has one source, so changing a rule changes the effect's size with it (DRY).	Each of the two constructors has one more parameter. Acceptable because only `WeatherInterpreter` calls them.
The effect classes stay free of rule constants, so they only describe what the effect does.	A caller could pass a different threshold from the one that chose the effect. Acceptable because `WeatherInterpreter` passes the same constant it tested.

Two interfaces, not one.

Design 1: one interface for both sides, with `reactAround(Location)` and `bringUpon(Actor, Location)`.

Pros	Cons
There is one interface for every weather effect. The interpreter could return the same type for today and tomorrow (KISS).	Every reaction would carry an omen method it cannot use, and every omen a reaction method (ISP).
	The unused bodies would do nothing or throw, so a caller could not trust every implementation (LSP risk). This is the Refused Bequest smell.

Design 2 (chosen): `WeatherReaction` (`reactAround(Location)`) and `ForecastOmen` (`bringUpon(Actor, Location)`). Both are chosen in the same `WeatherInterpreter`. Picking an effect for a reading is one job, whether the reading is for today or tomorrow.

Pros	Cons
Each class implements only the method it needs, so no effect carries a method it cannot honour (ISP).	There are two interfaces that look alike, which a reader must learn apart. Acceptable because they really differ: a reaction acts on a place, while an omen acts on the actor who asked.
`AskForecastAction` only sees a `ForecastOmen`. `Wanderer` and `ReactToWeatherAction` only see a `WeatherReaction`. None of them names a concrete effect class (DIP).	Both depend on `WeatherReading` (Connascence of Type), so a new field there touches both. Acceptable because `WeatherReading` is a small value object that rarely changes.

Effects that last several turns. The drought omen makes the asker lose 2 stamina a turn for 3 turns. It uses `DroughtWearinessStatus`, a `TimedStatus` (A2).

Design 1: spend stamina with `StatisticAdjuster.trySpend` and print nothing (our first version).

Pros	Cons
It reuses an existing, tested method. The status is only a few lines, and no A2 class changes.	`trySpend` is all-or-nothing, so a Farmer with 1 stamina lost nothing, against the proposal's "loses stamina every turn".
	The player saw no sign of the drain, because a status has no return value to report through.

Design 2 (chosen): a new `StatisticAdjuster.decrease`, the mirror of `increase`, drains what is there and stops at zero. The status overrides `TimedStatus.describeTurn` to print how much was drained and where.

Pros	Cons
The rule matches the proposal at every stamina level, because the drain stops at zero instead of being skipped.	`StatisticAdjuster`, an A2 class, gained one method. Acceptable because the new method only adds behaviour beside `increase`, and no existing method changed.

Pros	Cons
`decrease` returns a `StatisticChange`, so the report is worded the same way as every other stat change (DRY).	The status keeps the last change in a field between `applyEffect` and `describeTurn`. Acceptable because the field is private and overwritten each turn.
`describeTurn` is the hook `BurningStatus` already uses, so `TimedStatus` did not change (OCP).	

Foresight: a heatwave reaction above 35°C needs one new class, `HeatwaveReaction`, and one new `if` in `WeatherInterpreter`. `Wanderer` and the other reactions do not change.

Connascence: named constants avoid Connascence of Meaning. Passing the thresholds in removed Connascence of Value between the interpreter and two effect classes. Each action has Connascence of Type with one interface only. **Code smells:** avoided Switch Statements inside `Wanderer`, and magic numbers. Also avoided Speculative Generality (a rule engine) and Refused Bequest (reactions with an omen method). Accepted: one factory class names the six concrete classes.

D. Letting the Farmer ask about tomorrow

Design 1: give `Farmer` a method that searches the map for a Wanderer and asks it.

Pros	Cons
There is no new action class, and the Farmer's options sit in the Farmer's class.	The data (`WeatherSource`, `MapGeography`, `WeatherInterpreter`) belongs to the Wanderer, not the Farmer (Feature Envy).
	Any other actor that wanted to ask would copy the same search (DRY). The copies would have to stay the same (Connascence of Algorithm).

Design 2 (chosen): `Wanderer.allowableActions` offers an `AskForecastAction` to whoever stands next to it. This is the engine's normal way for one actor to offer an action to another. `ReactToWeatherAction` and `AskForecastAction` only pass the turn on to the reaction or omen they hold. The engine only offers actions from actors on the map. So `AskForecastAction` never needs to check that the Wanderer is still there.

Pros	Cons
It reuses the engine's protocol for one actor offering an action to another, like every other interactive entity (DRY).	Any adjacent actor is offered the action, not only the Farmer. Acceptable because only the player picks menu actions.
Each action has one job and names no concrete effect class (SRP).	`AskForecastAction` holds the same three collaborators as `Wanderer` (a small Data Clump). Acceptable because they are passed once, in `allowableActions`.
	`ReactToWeatherAction` only passes the turn on, close to a Middle Man. Acceptable because the engine needs an `Action` object for every turn.

Connascence: only Connascence of Name with the engine's existing API. **Code smells:** avoided Feature Envy (a Farmer method using the Wanderer's data). Accepted: `ReactToWeatherAction` and `UnreadableSkyAction` are close to Middle Men, because the engine needs an `Action` object.

E. Visibility, `final`, `protected` and `static`

Choice	Where	Why	Cost
Every class is `public`	All REQ5 classes	`game.weather` is meant to be reused by other features. Being `public` only makes reuse possible. Openness for extension comes from the interfaces and hooks, not from the modifier.	A wider public API, so more classes could depend on them (more connascence across packages).
Every field is `private`, and `final` where it never changes	All classes. `WeatherReading` is fully immutable	A reading is shared by the interpreter and the effect it creates, so it must not change.	`Wanderer.turnsSinceLastReaction` and `DroughtWearinessStatus.lastDrain` are the only changing fields. Each is changed by one method.
`protected` method	`OpenWeatherClient.fetch(URI)`	The one seam a test overrides to return saved replies without the network.	A subclass could replace the network call. The class comment says it exists for tests.
`protected` hook override	`DroughtWearinessStatus.describeTurn`	It overrides the `TimedStatus` hook, so only the template calls it.	Same-package classes could call it. None does.
`private` helpers	e.g. `OpenWeatherParser.readingFrom`, `WeatherInterpreter.isRainy`	They are steps, not contracts, so their signatures can change freely.	A subclass cannot reuse a single step. It must override the public method around it.
Constants are `private static final`	e.g. `STORM_WIND_THRESHOLD_MPS`, `MAX_EXTRA_TURNS`, `TIMEOUT`	A name explains each number, avoiding magic numbers (Connascence of Meaning). No public constants (A2 feedback).	Tests repeat the numbers.
No static methods	Every REQ5 class	A2 feedback penalised static methods as "function thinking".	`MapGeography` and `ApiKeyLoader` are objects passed in, not static utilities, so constructors take more parameters.

Connascence: `private` fields keep connascence on state inside each class. `WeatherReading` is immutable.
Code smells: avoided public attributes and missing modifiers.

F. Changes to the earlier design

REQ5 is new in A3. It builds on earlier classes and changes only three of them: `StackQueueValleyWorld`, `StatisticAdjuster` and `Plant`.

Earlier class	How REQ5 uses or changes it
`WanderingActor` (A2)	`Wanderer` extends it, so the unconscious check and wandering are reused.
`TemporaryGround` (REQ3)	`StormShelterGround` extends it, so the shelter restores the old ground with no new countdown code.
`Fire` (A2)	`CampfireReaction` lights the same `Fire` item.
`GrowthPausedStatus` (REQ1), `TimedStatus` (A2)	`FrostOmen` pauses plants. `DroughtWearinessStatus` reuses the countdown and the `describeTurn` hook.
`Plant.attemptGrowth` and `Plant.forceGrowth` (REQ1), changed	The pause check moved out of `Plant.tick` into both methods. So the frost also stops REQ4's Bounty (REQ1, Section D).
`StatisticAdjuster` (A2), changed	Gained `decrease`, the mirror of `increase`, for the drought. No existing method changed.
`Plantable`, `StrawberrySeedling` (REQ1)	`SowingOmen` plants through the same rule as everything else.
`StackQueueValleyWorld`, changed	Builds the client, the key loader and `MapGeography`, and places one Wanderer.

G. Known minor violations and why we accept them

Violation or cost	Why it is acceptable
The Wanderer can be burnt by its own campfire if it later walks onto a burning tile.	On purpose: `Fire` burns any actor and never asks who lit it, so it needs no type check. A Wanderer that stays somewhere cold pays for its fires, and it has 50 hit points.
`OpenWeatherClient` creates its own `HttpClient`, a small DIP gap.	Tests replace `fetch`, not the `HttpClient`, so injecting it would add a parameter nobody varies.
The weather condition is a `String` such as `"Rain"`, so the code depends on OpenWeatherMap's names (Connascence of Meaning).	The names are named constants in `WeatherInterpreter`, and the parser is the only class that reads them from JSON.
`AskForecastAction` holds the same three collaborators as `Wanderer` (a small Data Clump).	They are passed once, in `allowableActions`. A holder class for three fields would be a class used once.
Any actor next to the Wanderer is offered `AskForecastAction`, not only the Farmer.	Only the player picks menu actions. Other actors never choose it.
`WeatherInterpreter` names all six concrete classes, so a new reaction edits it. OCP is only partly met.	It is the one factory point (Section C). Everything else stays closed.
`ApiKeyLoader` and `OpenWeatherParser` turn errors into an empty result.	The game must keep running without weather. Tests cover the missing key, the missing file, the broken JSON and infinite values.

Summary

The final design uses the chosen design of every part. The game owns `WeatherSource`, and "no reading" is a normal answer (A). The place, the key and the reply each have one class (B). One interpreter picks between two narrow effect interfaces (C). Asking about tomorrow uses the engine's own protocol (D). Each abstraction either keeps the network out of the game and its tests, or separates effects the proposal already names. The main cost is more, smaller classes on the API side, and one factory class that names every effect. In return, the design passes a concrete OCP test: a Heatwave reaction needs one new class and one `if`. A different weather provider needs one new `WeatherSource` class. Neither changes `Wanderer`, `AskForecastAction`, the other effects or the parser's callers. Only `WeatherInterpreter` or `StackQueueValleyWorld` gains one line.

Principle	Where it shows	Change it absorbs	Where it is only partly met, and why
SRP	Each job has one class. `OpenWeatherClient` calls the API. `OpenWeatherParser` reads the JSON. `ApiKeyLoader` finds the key. `MapGeography` finds the place. `WeatherInterpreter` holds the rules.	A new JSON layout edits only the parser.	`Wanderer` both wanders and times its reactions. The timing is one counter, so a separate class would be used once.
OCP	Adding a reaction, omen or weather provider means writing one new class.	A Heatwave reaction or a cached weather source.	`WeatherInterpreter` gains one `if` per new rule (Section G).
LSP	Each of the six effects works in its action. `FixedWeatherSource` stands in for the real client in the Wanderer tests. Each effect has its own test class.	`FixedWeatherSource` replaces the real client in every test.	Every source must return an empty `Optional`, never `null`. The interface Javadoc says a missing reading is empty.
ISP	`WeatherReaction` and `ForecastOmen` each have one method. `WeatherSource` has the two the Wanderer needs.	A new omen implements one method and never sees reactions.	`WeatherSource` joins today and tomorrow. Every provider we know offers both, so splitting would add a type nobody uses alone.
DIP	`Wanderer` depends on `WeatherSource` and `WeatherReaction`. `AskForecastAction` on `ForecastOmen`. The client gets an `ApiKeyLoader` through its constructor, so tests can replace it. It is a concrete class, so this is injection, not full DIP.	A different provider that returns the same four values needs no change to the game classes.	`OpenWeatherClient` creates its own `HttpClient` (Section G).

DRY and KISS. The rules live in one interpreter. The countdowns reuse `TemporaryGround` and `TimedStatus`. The effects reuse `Fire`, `GrowthPausedStatus`, `Plantable` and `StatisticChange`. Six fixed rules are a few `if`s, not a rule engine. **Connascence.** The connascence we found between packages is of Name and Type, through `WeatherSource`, `WeatherReaction` and `ForecastOmen`. Each threshold is written once, in `WeatherInterpreter`. It passes the cold and frost thresholds to the effects that need them.

Testing. Each reaction, omen, action and API-layer class has its own test class, for example `WeatherInterpreterTest`, `WandererTest`, `OpenWeatherParserTest` and `ApiKeyLoaderTest`. They cover readings that meet or miss each rule, and which rule wins. They also cover extreme wind and cold, and a drought that drains the last stamina. Invalid input includes a missing key, broken JSON, negative wind and infinite values. No test uses the network: `FixedWeatherSource` replaces the client, and `OpenWeatherClientTest` overrides `fetch`. `mvn test` runs them all. `WandererStressHarness` is a separate tool, not part of `mvn test`. It plays many games with fake weather to look for rare crashes.